

INFORMATIQUE • 3ÈME

INFORMATIQUE – CLASSE DE 3ÈME

LEÇON N13

PRÉSENTATION DU LANGAGE DE DÉFINITION DES ALGORITHMES

DURÉE CONSEILLÉE : 1 SÉANCE (FICHE DE PROGRESSION HARMONISÉE)

DANS CETTE LEÇON

Cette leçon présente le langage de définition des algorithmes (pseudo-code) et la structure de base d'un algorithme : en-tête, déclarations, corps. L'élève apprend à rédiger des algorithmes clairs et non ambigus pour résoudre des situations concrètes de la vie courante.

- À la fin de cette leçon, je sais définir un algorithme et le langage de définition des algorithmes (pseudo-code)
- À la fin de cette leçon, je sais décrire la structure de base d'un algorithme (en-tête, partie déclarative, corps)
- À la fin de cette leçon, je sais identifier et nommer correctement les variables et constantes d'un problème
- À la fin de cette leçon, je sais distinguer les types de données élémentaires (entier, réel, caractère, chaîne, booléen)
- À la fin de cette leçon, je sais utiliser les instructions de base : lecture, écriture, affectation
- À la fin de cette leçon, je sais rédiger en pseudo-code un algorithme simple résolvant une situation concrète de la vie courante

TÉLÉCHARGE
ONBUCH

Gratuit • Google Play
Tuteur IA, annales,
concours, résultats



PASSE À ONBUCH+

Tous les cours signés, Tuteur IA
illimité, fiches PDF — onglet
OnBuch+ de l'app

CE COURS SIGNÉ CONTIENT

COURS

complet, illustré, pas à
pas

MÉTHODES

et exercices résolus

EXERCICES

type examen + corrigés

FICHE BILAN

l'essentiel à retenir



Signé • OnBuch+

Ludovic A.
CONFORME AU PROGRAMME OFFICIEL MINESEC

OnBuch+

SOMMAIRE

1. Rappels et notion de langage de définition des algorithmes
2. Structure de base d'un algorithme
3. Variables, constantes et types de données
4. Les instructions de base : lecture, écriture, affectation
5. Rédaction complète d'algorithmes sur des situations concrètes
6. Activité d'intégration
7. Méthodes et exercices résolus
8. Exercices
9. Corrigés des exercices
10. Fiche bilan

OBJECTIFS DE LA LEÇON

- À la fin de cette leçon, je sais définir un algorithme et le langage de définition des algorithmes (pseudo-code)
- À la fin de cette leçon, je sais décrire la structure de base d'un algorithme (en-tête, partie déclarative, corps)
- À la fin de cette leçon, je sais identifier et nommer correctement les variables et constantes d'un problème
- À la fin de cette leçon, je sais distinguer les types de données élémentaires (entier, réel, caractère, chaîne, booléen)
- À la fin de cette leçon, je sais utiliser les instructions de base : lecture, écriture, affectation
- À la fin de cette leçon, je sais rédiger en pseudo-code un algorithme simple résolvant une situation concrète de la vie courante
- À la fin de cette leçon, je sais justifier la démarche suivie et vérifier la correction d'un algorithme par un test d'exécution à la main

PRÉREQUIS

- Notion d'algorithme vue en classe de 4ème (suite ordonnée d'instructions)
- Notion de variable et de donnée en informatique
- Opérations arithmétiques élémentaires (addition, soustraction, multiplication, division)

- Résolution de problèmes simples de la vie courante (calcul de moyenne, de prix, de périmètre)
- Notion d'ordinateur et de programme (unité 1)

1 RAPPELS ET NOTION DE LANGAGE DE DÉFINITION DES ALGORITHMES

Mama Ngo Bell veut expliquer à son neveu, qui vit à Bafoussam, comment préparer son célèbre poulet DG. Elle peut lui décrire la recette en français, en ewondo ou en pidgin : le résultat sera le même si les étapes sont claires et dans le bon ordre. « D'abord tu découpes le poulet, ensuite tu le fais frire, puis tu ajoutes les légumes... » Peu importe la langue utilisée : ce qui compte, c'est la suite d'étapes. De même, en informatique, un algorithme peut être décrit en français courant, mais pour éviter toute ambiguïté et se rapprocher de l'ordinateur, on utilise un langage spécial appelé **langage de définition des algorithmes** (ou **pseudo-code**). Comment écrire correctement un algorithme dans ce langage normalisé ? C'est l'objet de cette leçon.

1.1 Rappel : qu'est-ce qu'un algorithme ?

Avant d'apprendre un nouveau langage, rappelons d'abord ce que nous cherchons à exprimer avec lui.

DÉFINITION Algorithme

Un **algorithme** est une suite finie et ordonnée d'instructions élémentaires permettant de résoudre un problème ou d'accomplir une tâche précise.

Décomposons cette définition, car chaque mot y compte :

- **Suite finie** : l'algorithme doit s'arrêter après un nombre limité d'étapes. Une recette qui ne se termine jamais ne sert à rien !
- **Ordonnée** : les instructions s'exécutent dans un ordre précis. On ne peut pas « ajouter les légumes » avant d'avoir « découpé le poulet ».
- **Instructions élémentaires** : chaque étape doit être simple, claire et réalisable sans réflexion supplémentaire.
- **Résoudre un problème** : un algorithme répond toujours à un besoin précis (calculer une moyenne, trouver un itinéraire, effectuer un retrait d'argent).

LESAVAIS-TU ? D'où vient le mot « algorithme » ?

Le mot algorithme vient du nom du mathématicien perse Al-Khawarizmi, qui vécut au IX^e siècle à Bagdad. Dans ses ouvrages, il décrivait des méthodes de calcul étape par étape, par exemple pour résoudre des équations. Son nom, latinisé en Algoritmi, est devenu le mot que nous utilisons aujourd'hui dans le monde entier. Plus de mille ans après lui, chaque téléphone mobile, chaque guichet automatique et chaque application exécute des millions d'algorithmes chaque jour !

1.2 Des algorithmes dans la vie courante

Tu exécutes des algorithmes tous les jours sans t'en rendre compte. En voici trois exemples tirés de la vie quotidienne au Cameroun.

Exemple 1 : la recette de cuisine. Préparer un poulet DG, c'est suivre un algorithme : découper le poulet, assaisonner, faire frire, préparer les légumes, mélanger, laisser mijoter, servir. Chaque étape est élémentaire et l'ordre est essentiel.

Exemple 2 : l'itinéraire. Expliquer à un ami comment aller de la gare de Bafoussam au marché central : « Sors de la gare, tourne à droite, marche jusqu'au rond-point, prends la deuxième sortie, le marché est sur ta gauche. » C'est une suite ordonnée d'instructions.

Exemple 3 : le retrait d'argent au guichet mobile (OM/MoMo). Quand tu retires de l'argent avec Orange Money ou MTN Mobile Money, le téléphone exécute un algorithme :

Composer le code du service (#150# ou *126#)
Choisir "Retrait d'argent"
Saisir le montant à retirer
Saisir le code secret
Confirmer l'opération
Recevoir le message de confirmation

Remarque que si tu saisis un mauvais code secret, l'opération est refusée : l'algorithme prévoit même les cas d'erreur !

À RETENIR L'essentiel

Un algorithme n'est pas forcément lié à l'ordinateur : toute méthode précise, décrite étape par étape, pour accomplir une tâche est un algorithme. L'ordinateur n'est qu'une machine capable d'exécuter des algorithmes très rapidement et sans se tromper.

1.3 Les limites du langage naturel

Sil'on peut décrire un algorithme en français courant, pourquoi inventer un autre langage ? Parce que le langage naturel (le français de tous les jours) présente de sérieux défauts lorsqu'il s'agit de donner des instructions à une machine.

- **L'ambiguïté** : une même phrase peut avoir plusieurs sens. Si l'on écrit « ajoute le sel et le poivre à la fin », faut-il ajouter le poivre à la fin seulement, ou les deux ? Un humain devine ; un ordinateur, non.
- **La longueur** : décrire en phrases complètes un calcul simple prend beaucoup de place et de temps. « Tu prends la somme des deux notes, tu la divises par deux, et tu obtiens la moyenne » est bien plus long que « moyenne $\leftarrow (note1 + note2) / 2$ ».
- **L'imprécision** : des mots comme « un peu », « environ », « bientôt » n'ont pas de signification exacte pour une machine. L'ordinateur a besoin de valeurs et d'actions parfaitement définies.

EXEMPLE Une consigne ambiguë

Le père de famille dit à son fils : « Va acheter du pain, et s'il y a des œufs, prends-en six. »
Le fils revient avec six pains ! Pourquoi ? Parce que la phrase est ambiguë : « en » peut se rapporter aux œufs (le sens voulu) ou au pain (le sens compris). Un algorithme bien écrit lèverait cette ambiguïté : SI il y a des œufs ALORS acheter 6 œufs.

1.4 Le langage de définition des algorithmes (pseudo-code)

Pour décrire un algorithme de façon claire, courte et sans ambiguïté, les informaticiens ont créé un langage intermédiaire, à mi-chemin entre le français et les langages de programmation.

DÉFINITION Langage de définition des algorithmes

Le **langage de définition des algorithmes** (LDA), appelé aussi **pseudo-code**, est un langage conventionnel, proche du français, utilisé pour décrire un algorithme de manière précise et non ambiguë, indépendamment de tout langage de programmation.

Le pseudo-code possède trois caractéristiques essentielles :

- **Des mots-clés réservés** : certains mots ont un sens fixe et ne peuvent pas être utilisés autrement. Par exemple : Algorithme, Début, Fin, Lire, Écrire, Si, Alors. Ces mots sont « réservés » : on ne peut pas s'en servir pour nommer autre chose.
- **Une syntaxe précise** : chaque instruction obéit à une forme exacte. On écrit Lire(note) et non « lis un peu la note si tu veux ». Cette rigueur supprime toute ambiguïté.
- **L'indépendance vis-à-vis des langages de programmation** : le pseudo-code n'appartient à aucun langage (ni C, ni JavaScript, ni Python). Un même algorithme écrit en pseudo-code pourra ensuite être traduit dans n'importe quel langage de programmation, exactement comme la recette de Mama Ngo Bell peut être racontée en français, en ewondo ou en pidgin.

EXEMPLE Premier algorithme en pseudo-code

Voici comment on écrit, en pseudo-code, l'algorithme qui calcule la moyenne de deux notes d'un élève de 3^e :

```
Algorithme Moyenne
Variable note1, note2, moyenne : reel
Debut
    Écrire("Entrez la premiere note : ")
    Lire(note1)
    Écrire("Entrez la deuxieme note : ")
    Lire(note2)
```

```
moyenne <- (note1 + note2) / 2
Ecrire("La moyenne est :", moyenne)
Fin
```

On reconnaît les mots-clés réservés **Algorithme**, **Variable**, **Debut**, **Fin**, **Lire** et **Ecrire**. Nous étudierons chacun d'eux en détail dans les sections suivantes de cette leçon.

1.5 De l'idée à la machine : la chaîne de traduction

L'ordinateur ne comprend ni le français ni le pseudo-code : il ne comprend que le **langage machine**, fait de 0 et de 1. Pour passer d'un problème concret à son exécution par l'ordinateur, on suit donc une chaîne en quatre étapes :

1. **Le problème** : on part d'une situation de la vie réelle (calculer les moyennes d'une classe, gérer les ventes d'une boutique).
2. **L'algorithme** : par analyse du problème, on conçoit la suite d'étapes qui le résout, et on la rédige en pseudo-code.
3. **Le programme** : on traduit l'algorithme dans un langage de programmation (C, JavaScript, Python...). C'est le travail du programmeur.
4. **Le résultat** : l'ordinateur exécute le programme et produit le résultat attendu.



La chaîne de traduction : du problème réel au résultat produit par l'ordinateur.

ATTENTION Ne confonds pas algorithme et programme !

C'est l'erreur la plus fréquente en classe de 3^e. L'algorithme est la méthode de résolution, écrite en pseudo-code, indépendante de toute machine. Le programme est la traduction de cet algorithme dans un langage de programmation précis (C, JavaScript...), exécutable par l'ordinateur. Retiens l'image : l'algorithme est la recette écrite sur le cahier ; le programme est le plat cuisiné dans une cuisine précise, avec des ustensiles précis.

EXERCICE RÉSOLU 1 Identifier les étapes de la chaîne de traduction

Énoncé. Le proviseur d'un collège de Douala veut un outil qui affiche, pour chaque élève, la mention obtenue au BEPC à partir de sa moyenne. Un informaticien rédige d'abord la

méthode en pseudo-code, puis l'écrit en langage C, et l'ordinateur affiche finalement « Mention : Bien » pour une moyenne de 14. Pour chaque élément suivant, indique à quelle étape de la chaîne de traduction il correspond : (a) la moyenne de l'élève et sa mention ; (b) le texte en pseudo-code ; (c) le code en langage C ; (d) le besoin du proviseur.

SOLUTION

Solution détaillée. On applique la chaîne : problème → algorithme → programme → résultat.

- (a) La moyenne et la mention affichée constituent le résultat, produit par l'exécution du programme.
- (b) Le texte en pseudo-code est l'algorithme, obtenu par l'analyse du problème.
- (c) Le code en langage C est le programme, obtenu par traduction de l'algorithme.
- (d) Le besoin du proviseur est le problème de départ, la situation concrète à résoudre.

Conclusion : chaque étape de la chaîne a un rôle précis, et aucune ne peut être sautée : sans analyse, pas d'algorithme ; sans algorithme, pas de programme fiable.

EXERCICE 1 À toi de jouer

1. Donne deux exemples d'algorithmes de la vie courante, différents de ceux du cours, en précisant les étapes dans l'ordre. 2. Pourquoi le français courant est-il mal adapté pour donner des instructions à un ordinateur ? Cite deux raisons. 3. Quelle est la différence entre un algorithme et un programme ? 4. Remets dans l'ordre les étapes de la chaîne de traduction : programme, problème, résultat, algorithme.

CORRIGÉ Exercice — À toi de jouer

1. Exemples possibles : la lessive (trier le linge, tremper, savonner, rincer, essorer, étendre) ; la recharge de crédit téléphonique (composer le code, saisir le numéro de la carte, valider, recevoir la confirmation). 2. Le français courant est ambigu (une phrase peut avoir plusieurs sens) et imprécis (mots vagues comme « un peu »), alors qu'un ordinateur exige des instructions exactes. Il est aussi trop long. 3. L'algorithme est la méthode de résolution écrite en pseudo-code ; le programme est sa traduction dans un langage de programmation, exécutable par l'ordinateur. 4. Ordre correct : problème → algorithme → programme → résultat.

À RETENIR Bilan de la section

Un algorithme est une suite finie et ordonnée d'instructions élémentaires qui résout un problème. Pour le décrire sans ambiguïté, on utilise le langage de définition des algorithmes (pseudo-code), un langage conventionnel proche du français, doté de mots-clés réservés et d'une syntaxe précise, indépendant de tout langage de programmation. La chaîne de traduction mène du problème au résultat : analyse, pseudo-code, programme, exécution. Dans la prochaine section, nous découvrirons la structure de base d'un algorithme écrit en pseudo-code.

2 STRUCTURE DE BASE D'UN ALGORITHME

Dans la section précédente, nous avons vu qu'un algorithme est une suite finie et ordonnée d'instructions permettant de résoudre un problème, et que pour le rédiger sans ambiguïté, on utilise un **langage de définition des algorithmes**, appelé aussi **pseudo-code**. Mais une question se pose immédiatement : comment ce texte doit-il être organisé ? Peut-on écrire les instructions n'importe comment, dans le désordre ? Non. Tout comme une lettre administrative comporte toujours un en-tête, un corps et une signature, un algorithme suit une **structure** précise et immuable. C'est cette structure que nous allons découvrir et maîtriser dans cette section.

DÉFINITION Langage de définition des algorithmes (Pseudo-code)

Un **langage de définition des algorithmes** (ou **pseudo-code**) est un langage normalisé, indépendant de toute machine, qui permet d'écrire des algorithmes avec une syntaxe précise, rigoureuse et compréhensible par tout informaticien. Il utilise des mots-clés en français (Algorithme, Var, Debut, Fin, Si, TantQue, ...) pour décrire la logique du traitement sans se soucier des détails syntaxiques d'un langage de programmation réel (C, Python, JavaScript...).

2.1 Les trois parties d'un algorithme

Intuition : à quoi sert une structure ?

Imagine que tu demandes à un ami de préparer du ndolé. Si tu lui donnes les étapes dans le désordre (« mets le sel, lave les feuilles, achète les arachides, sers le plat »), le résultat sera un désastre. De même, l'ordinateur (ou la personne qui lit ton algorithme) a besoin de savoir avant de commencer : de quoi on parle (le nom du problème), quels ingrédients on va utiliser (les variables), et seulement ensuite ce qu'on fait avec (les instructions).

PROPRIÉTÉ Structure générale d'un algorithme

Tout algorithme écrit en pseudo-code comporte trois parties, toujours dans cet ordre :

1. L'en-tête : il donne le nom de l'algorithme, introduit par le mot-clé `Algorithme`;
2. La partie déclarative : elle annonce toutes les variables et constantes utilisées, avec leurs types, après les mots-clés `Var` et `Const`;
3. Le corps : il contient les instructions à exécuter, délimité par les mots-clés `Debut` et `Fin`.

Cette propriété est admise : elle constitue la convention du langage de définition des algorithmes utilisé au collège. Nous la justifierons par l'exemple tout au long de la leçon.



Les trois parties d'un algorithme : l'en-tête (le nom), la partie déclarative (les objets utilisés) et le corps (les actions).

2.2 L'en-tête : nommer son algorithme

L'**en-tête** est la première ligne de l'algorithme. Sa syntaxe est très simple :

PROPRIÉTÉ Syntaxe de l'en-tête

L'en-tête s'écrit :

`Algorithme nom_de_l_algorithme`

Le mot-clé `Algorithme` est obligatoire et s'écrit avec la majuscule. Il est suivi du **nom** de l'algorithme, qui est un **identificateur**.

Un **identificateur** est un nom choisi par le programmeur pour désigner un algorithme, une variable ou une constante. On ne peut pas choisir n'importe quel nom : il faut respecter des règles

strictes.

MÉTHODE Bien choisir un identificateur

Pour nommer un identificateur (algorithme, variable, constante), respecte les règles suivantes :

1. Il commence obligatoirement par une lettre (jamais par un chiffre) ;
2. Il ne contient ni espace, ni accent, ni caractère spécial (pas de é, ç, -, ?, ! ...) ; seul le tiret bas _ est toléré ;
3. Il est court et significatif : il doit rappeler ce que l'objet représente.

Exemples corrects : Somme, moyenne, prixUnitaire, age, note1.

Exemples incorrects : 1note (commence par un chiffre), prix unitaire (espace), moyenné (accent), nbre-eleves (tiret).

ATTENTION Erreurs fréquentes sur les identificateurs

Un nom comme `x` ou `truc` est syntaxiquement correct, mais il ne veut rien dire : trois jours plus tard, tu ne sauras plus à quoi il sert ! Préfère `moyenneClasse` à `m`. À l'inverse, évite les noms trop longs comme `lamoyennedeselevesdelaclassedetroisieme`. L'équilibre : court et clair.

2.3 La partie déclarative : annoncer les objets utilisés

Intuition. Avant de cuisiner, on sort tous les ingrédients sur la table. Avant d'exécuter les instructions, l'algorithme doit annoncer tous les objets qu'il va manipuler : c'est le rôle de la **partie déclarative**.

DÉFINITION Partie déclarative

La partie déclarative est la zone de l'algorithme où l'on déclare (annonce) toutes les **variables** et toutes les **constantes** qui seront utilisées dans le corps, en précisant leur **type** (entier, réel, caractère, chaîne, booléen). Elle est introduite par les mots-clés `Var` (pour les variables) et `Const` (pour les constantes).

PROPRIÉTÉ Syntaxe des déclarations

- Variables : `Var nom1, nom2, ... : type`
- Constantes : `Const nom = valeur`

On peut regrouper sur une même ligne plusieurs variables de même type, séparées par des virgules. Une constante reçoit sa valeur dès sa déclaration et cette valeur ne changera jamais pendant l'exécution. Les types de base sont : entier, reel, caractere, chaine, booleen.

EXEMPLE Une partie déclarative

```
Var a, b, s : entier  
Var moyenne : reel  
Var nomEleve : chaine  
Const TAUX = 18
```

Ici, on annonce trois variables entières (a, b, s), une variable réelle (moyenne), une chaîne de caractères (nomEleve) et une constante TAUX qui vaudra toujours 18 (par exemple un taux de TVA en pourcentage).

2.4 Le corps : les instructions entre Debut et Fin

Le **corps** de l'algorithme est la partie « active » : c'est là que se trouvent les **instructions** (lire des données, calculer, afficher des résultats). Ces instructions seront étudiées en détail dans la section 4 ; ici, retenons surtout leur cadre.

PROPRIÉTÉ Syntaxe du corps

Le corps est obligatoirement délimité par les mots-clés **Debut** et **Fin** :

- **Debut** marque le point de départ de l'exécution ;
- **Fin** marque l'arrêt de l'algorithme.

Les instructions situées entre ces deux mots-clés sont exécutées dans l'ordre où elles sont écrites, de haut en bas.

ATTENTION Les deux erreurs classiques du débutant

- Utiliser une variable non déclarée : si tu écris $s \leftarrow a + b$ sans avoir déclaré **s** dans la partie **Var**, l'algorithme est faux. Toute variable utilisée dans le corps doit avoir été annoncée avant.
- Oublier **Debut** ou **Fin** : sans ces mots-clés, on ne sait plus où commence et où s'arrête l'algorithme. Vérifie toujours que chaque **Debut** a bien son **Fin**.

2.5 Commentaires et indentation : rendre l'algorithme lisible

Un algorithme n'est pas écrit seulement pour l'ordinateur : il est écrit pour être lu par des humains (toi, ton camarade, ton professeur). Deux outils rendent un algorithme lisible.

DÉFINITION Commentaire

Un **commentaire** est un texte libre placé dans l'algorithme pour expliquer ce que fait une ligne ou un bloc d'instructions. Il est ignoré lors de l'exécution. On l'introduit par `//` : tout ce qui suit sur la ligne est un commentaire.

DÉFINITION Indentation

L'**indentation** est le décalage vers la droite (avec des espaces) des instructions situées à l'intérieur d'un bloc. Elle permet de voir d'un coup d'œil ce qui se trouve entre `Debut` et `Fin`.

Compare les deux versions suivantes : elles font exactement la même chose, mais laquelle préfères-tu lire ?

```
Algorithme Somme
Var a, b, s : entier
Debut
Lire(a, b)
s <- a + b
Ecrire(s)
Fin
```

```
Algorithme Somme
// Calcule et affiche la somme de deux nombres
Var a, b, s : entier // les deux nombres et leur somme
Debut
  Lire(a, b) // saisie des deux valeurs
  s <- a + b // calcul de la somme
  Ecrire(s) // affichage du resultat
Fin
```

La seconde version, indentée et commentée, est incomparablement plus claire. Prends dès maintenant l'habitude d'indenter et de commenter : c'est un **savoir-être** du bon programmeur.

2.6 Exemple complet commenté : la somme de deux nombres

EXEMPLE Algorithme Somme, complet et commenté

```
| Algorithme Somme
```

```
// Role : lit deux nombres entiers et affiche leur somme
Var a, b, s : entier // a et b : nombres saisis ; s : somme
Debut
    Lire(a, b) // l'utilisateur entre les deux valeurs
    s <- a + b // on range le resultat de a + b dans s
    Ecrire(s) // on affiche le contenu de s
Fin
```

On retrouve bien les trois parties : l'en-tête (Algorithme Somme), la partie déclarative (Var a, b, s : entier) et le corps (entre Debut et Fin).

EXERCICE RÉSOLU 2 Identifier les trois parties d'un algorithme

Énoncé. Voici un algorithme écrit par un élève de 3ème :

```
Algorithme Double
Var n, d : entier
Debut
    Lire(n)
    d <- 2 * n
    Ecrire(d)
Fin
```

1) Indique l'en-tête, la partie déclarative et le corps. 2) Que fait cet algorithme si l'utilisateur saisit la valeur 7 ?

SOLUTION

Solution. 1) L'en-tête est la ligne Algorithme Double. La partie déclarative est la ligne Var n, d : entier : elle annonce deux variables entières. Le corps est l'ensemble des lignes comprises entre Debut et Fin, soit les trois instructions Lire(n), d <- 2 * n et Ecrire(d).

2) Traçons l'exécution pas à pas avec $n = 7$:

- Lire(n) : la variable n reçoit la valeur 7 ;
- d <- 2 * n : la variable d reçoit $2 \times 7 = 14$;
- Ecrire(d) : l'algorithme affiche 14.

Cet algorithme calcule et affiche donc le double du nombre saisi, ce qui justifie le nom bien choisi de l'identificateur Double.

EXERCICE 2 À toi de jouer

Un élève veut écrire un algorithme qui calcule le périmètre d'un carré à partir de la longueur de son côté. Il écrit :

```
Algorithme perimetre-carre
Debut
```

```
Lire(c)
p <- 4 * c
Ecrire(p)
Fin
```

Trouve les trois erreurs commises (indice : regarde l'identificateur, la partie déclarative et les mots-clés), puis réécris l'algorithme correctement, avec un commentaire et une indentation soignée.

CORRIGÉ Exercice 1

Les trois erreurs sont :

1. L'identificateur `perimetre-carre` est incorrect : il contient un tiret. Il faut écrire par exemple `PerimetreCarre` (pas d'accent sur `Perimetre` non plus).
2. La partie déclarative est absente : les variables `c` et `p` ne sont pas déclarées.
3. L'absence de déclaration rend le corps invalide ; on doit ajouter `Var c, p : reel` avant `Debut`.

Version corrigée :

```
Algorithme PerimetreCarre
// Calcule le perimetre d'un carre a partir de son cote
Var c, p : reel          // c : cote du carre ; p : perimetre
Debut
  Lire(c)                // saisie du cote
  p <- 4 * c              // calcul du perimetre
  Ecrire(p)              // affichage du resultat
Fin
```

À RETENIR L'essentiel de la section

- Tout algorithme comporte trois parties : l'en-tête (`Algorithme` + nom), la partie déclarative (`Var` / `Const` + types) et le corps (entre `Debut` et `Fin`).
- Un identificateur commence par une lettre, sans espace, ni accent, ni caractère spécial ; il doit être court et significatif.
- Toute variable utilisée dans le corps doit avoir été déclarée au préalable.
- Les commentaires (`//`) et l'indentation rendent l'algorithme lisible sans modifier son exécution.

Maintenant que la charpente est bien en place, il reste à donner vie au corps de l'algorithme : comment lire une donnée, afficher un résultat, ranger une valeur dans une variable ? C'est l'ob-

jet des sections suivantes, qui détaillent les instructions de base.

3 VARIABLES, CONSTANTES ET TYPES DE DONNÉES

Dans la section précédente, nous avons découvert la structure de base d'un algorithme : un en-tête qui donne son nom, une partie **déclarative** où l'on annonce les objets utilisés, puis un corps délimité par `Début` et `Fin` contenant les instructions. Mais que déclare-t-on exactement dans cette partie déclarative ? Pour le comprendre, posons-nous une question simple.

Imagine que le proviseur de ton collège te demande d'écrire un algorithme qui calcule la moyenne d'un élève à partir de ses notes. Pour faire ce calcul, l'algorithme doit retenir quelque part les notes, la somme, puis la moyenne. Or un ordinateur ne retient rien « dans sa tête » : il stocke les informations dans sa **mémoire**. Il faut donc réserver des emplacements dans cette mémoire, leur donner des noms pour pouvoir les retrouver, et préciser quelle sorte d'information chacun contiendra. C'est exactement le rôle des **variables**, des **constantes** et des **types de données**, que nous allons étudier dans cette section.

3.1 La notion de variable

Intuition : la boîte étiquetée.

Pense à une boîte sur laquelle tu colles une étiquette, par exemple « age ». À l'intérieur de la boîte, tu places une fiche sur laquelle est écrit un nombre : 14. L'année suivante, tu peux ouvrir la boîte, retirer la fiche et en mettre une nouvelle portant le nombre 15. La boîte garde le même nom (« age »), mais son contenu a changé. Une variable en algorithmique fonctionne exactement comme cette boîte étiquetée : c'est un emplacement de la mémoire de l'ordinateur, identifié par un nom, et dont le contenu peut être modifié au cours de l'exécution de l'algorithme.

DÉFINITION Variable

Une **variable** est un emplacement de la mémoire de l'ordinateur, identifié par un nom (appelé identificateur) et contenant une valeur qui peut varier (changer) au cours de l'exécution de l'algorithme.

Une variable possède donc trois caractéristiques essentielles, qu'il faut toujours avoir en tête :

- son **nom** (ou identificateur) : c'est l'étiquette qui permet de désigner l'emplacement mémoire, par exemple `age`, `moyenne`, `nomEleve` ;
- son **type** : c'est la nature des valeurs qu'elle peut contenir (nombre entier, nombre décimal, texte, etc.) ;
- sa **valeur** : c'est le contenu actuel de l'emplacement mémoire, qui peut changer pendant l'exécution.

PROPRIÉTÉ Règles de choix d'un nom de variable

Le nom d'une variable (identificateur) doit respecter les règles suivantes :

- il commence obligatoirement par une lettre ;
- il ne contient ni espace, ni accent, ni caractère spécial (seuls les lettres, les chiffres et parfois le tiret bas `_` sont admis) ;
- il doit être différent des mots réservés du langage (Algorithme, Var, Début, Fin, etc.) ;
- il doit être explicite : on préfère `prixHT` à `x`, car un bon nom rend l'algorithme plus facile à lire et à comprendre.

ATTENTION Noms de variables interdits ou déconseillés

Les déclarations suivantes sont incorrectes : `2note` (commence par un chiffre), `prix TTC` (contient un espace), `prénom` (contient un accent), `Début` (mot réservé du langage). Choisissez toujours des noms simples, sans espace ni accent, et qui décrivent clairement le contenu : `note2`, `prixTTC`, `prenom`.

3.2 La notion de constante

Intuition : ce qui ne change jamais.

Dans certaines situations, une valeur reste toujours la même pendant tout le déroulement du programme. Par exemple, au Cameroun, le taux de la Taxe sur la Valeur Ajoutée (TVA) est fixé à 19,25 %. Si tu écris un algorithme qui calcule le prix TTC des articles vendus dans la boutique de ta famille, ce taux sera le même pour tous les calculs : il n'a aucune raison de changer pendant l'exécution. Plutôt que de réécrire le nombre 19,25 partout (avec le risque de se tromper une fois sur dix), on le range une seule fois dans une **constante**.

DÉFINITION Constante

Une **constante** est un emplacement de la mémoire de l'ordinateur, identifié par un nom, dont la valeur est fixée au départ et ne peut pas changer pendant l'exécution de l'algorithme.

L'intérêt des constantes est double :

- **lisibilité** : écrire `TAUX` est plus parlant qu'écrire 19,25 au milieu d'un calcul ;
- **facilité de modification** : si le taux de TVA change un jour, il suffit de modifier une seule ligne (la déclaration de la constante) au lieu de chercher le nombre dans tout l'algorithme.

EXEMPLE Une constante en situation réelle

Pour calculer le prix toutes taxes comprises (TTC) d'un article à partir de son prix hors taxes (HT), on déclare :

```
Const TAUX = 19,25  
Var prixHT, prixTTC : reel
```

Le calcul s'écrira ensuite : $\text{prixTTC} \leftarrow \text{prixHT} + \text{prixHT} * \text{TAUX} / 100$. La valeur de TAUX reste fixe pendant toute l'exécution, tandis que prixHT et prixTTC peuvent changer d'un article à l'autre.

À RETENIR Variable ou constante ?

Si la valeur d'une donnée peut changer pendant l'exécution (une note saisie, un total qui s'accumule, un âge), on utilise une variable. Si la valeur est fixée une fois pour toutes (taux de TVA, nombre de jours d'une semaine, valeur de π), on utilise une constante. Par convention, on écrit souvent les noms des constantes en majuscules (TAUX, PI, NB_JOURS) pour les distinguer facilement des variables.

3.3 Syntaxe de déclaration

Avant d'utiliser une variable ou une constante dans le corps d'un algorithme, il faut obligatoirement l'**annoncer** dans la partie déclarative : c'est ce qu'on appelle la **déclaration**. Déclarer, c'est dire à l'ordinateur : « réserve-moi une boîte, voici son nom, et voici la nature de ce qu'elle contiendra ».

PROPRIÉTÉ Syntaxe de déclaration

La déclaration d'une variable s'écrit :

```
Var nom_de_la_variable : type
```

On peut déclarer plusieurs variables de même type sur une seule ligne, en séparant leurs noms par des virgules :

```
Var note1, note2, note3 : reel
```

La déclaration d'une constante s'écrit :

```
Const NOM_DE_LA_CONSTANTE = valeur
```

Le type d'une constante n'est pas précisé : il est déterminé automatiquement par la valeur qu'on lui donne.

EXEMPLE Partie déclarative complète

Voici la partie déclarative d'un algorithme qui gère les informations d'un élève :

```
Algorithme Bulletin
Const TAUX = 19,25
Const NB_TRIMESTRES = 3
Var nom : chaîne
Var age : entier
Var moyenne, total : reel
Var admis : booleen
Debut
    (* instructions *)
Fin
```

Remarque l'ordre : on déclare d'abord les constantes (Const), puis les variables (Var), avant le mot Debut.

3.4 Les types élémentaires de données

Le **type** d'une variable indique la **nature** des valeurs qu'elle peut contenir. C'est une information indispensable pour l'ordinateur : selon le type, il ne réserve pas la même place en mémoire et n'autorise pas les mêmes opérations (on peut additionner deux nombres, mais pas deux pré-noms!). En classe de 3^e, nous utilisons cinq **types élémentaires**.

DÉFINITION Types élémentaires

Les cinq types élémentaires du langage algorithmique sont :

- **entier** : un nombre sans partie décimale, positif ou négatif. Exemples : 15, -3, 2024.
- **reel** : un nombre qui peut avoir une partie décimale. Exemples : 3,14, 12,5, -0,75.
- **caractere** : un seul caractère (lettre, chiffre ou symbole), écrit entre apostrophes. Exemples : 'A', 'b', '7', '?'.
Exemples : 'A', 'b', '7', '?'.
- **chaîne** : une suite de caractères (un texte), écrite entre guillemets. Exemples : "Douala", "Amina", "3e C".
- **booleen** : une valeur de vérité, qui ne peut être que Vrai ou Faux. Sert à représenter une réponse de type oui/non.

Le tableau suivant récapitule ces types avec des exemples tirés de la vie scolaire.

Type	Exemples de valeurs	Exemple d'utilisation
entier	15 ; 2024 ; -3	un âge, un nombre d'élèves, une année
reel	3,14 ; 12,5 ; 19,25	un prix, une moyenne, une taille en mètres
caractere	'A' ; 'g' ; '7'	une seule lettre, la mention d'un bulletin
chaine	"Douala" ; "Amina"	un nom, une adresse, une classe
booléen	Vrai ; Faux	« l'élève est-il admis ? », « le stock est-il vide ? »

ATTENTION Apostrophes ou guillemets ?

Ne confonds pas le type `caractere` et le type `chaine` : un caractère seul s'écrit entre apostrophes ('A'), tandis qu'une chaîne s'écrit entre guillemets ("Amina"). Attention aussi : '7' est un caractère (le symbole 7), "7" est une chaîne, et 7 est un entier : ce sont trois données de types différents, avec lesquelles on ne peut pas faire les mêmes opérations.

Pour bien visualiser ce qui se passe en mémoire après les déclarations et les premières affectations, représentons nos « boîtes étiquetées » :



Chaque boîte correspond à un emplacement de la mémoire. L'étiquette orange est le nom de la variable, le contenu de la boîte est sa valeur actuelle, et le type précise la nature de ce contenu. Pendant l'exécution, on pourra remplacer le contenu de la boîte `age` par 15, celui de la boîte `moyenne` par 13,75, etc. En revanche, on ne pourra jamais mettre un texte dans la boîte `age` : son type `entier` l'interdit.

3.5 Choisir le bon type pour une donnée

Face à une situation concrète, la première question à se poser avant de déclarer une variable est : « quelles valeurs cette donnée peut-elle prendre ? ». La réponse détermine le type.

MÉTHODE Comment choisir le type d'une donnée

Pour choisir correctement le type d'une variable, procède par étapes :

1. Identifie la donnée à mémoriser (par exemple : l'âge d'un élève).
2. Demande-toi quelles valeurs elle peut prendre :
 - un nombre sans virgule (14, 15, 16...) → type `entier`;
 - un nombre qui peut avoir une virgule (12,5 ; 999,99...) → type `reel`;
 - une seule lettre ou un seul symbole ('A', 'F'...) → type `caractere`;
 - un texte de plusieurs caractères ("Amina", "Yaoundé"...) → type `chaine`;
 - une réponse de type oui/non → type `booléen`.
3. Vérifie les cas limites : la valeur peut-elle un jour devenir décimale ? Si oui, choisis `reel` même si les premiers exemples sont des entiers.
4. Déclare la variable avec le type retenu : `Var age : entier.`

Appliquons cette méthode à quelques données de la vie courante :

Donnée	Type choisi	Justification
L'âge d'un élève	<code>entier</code>	on exprime l'âge en années entières : 14, 15...
Le prix d'un cahier	<code>reel</code>	un prix peut être décimal : 350,50 FCFA
Le nom d'un élève	<code>chaine</code>	c'est un texte de plusieurs caractères
La moyenne générale	<code>reel</code>	une moyenne est rarement un nombre entier : 12,5
Le sexe (M ou F)	<code>caractere</code>	une seule lettre : 'M' ou 'F'
L'élève est admis ?	<code>booléen</code>	la réponse est Vrai ou Faux

ATTENTION Erreur fréquente : le prix ou la moyenne déclarés en entier

Une erreur très classique consiste à déclarer un prix ou une moyenne comme `entier` parce que « dans mon exemple, c'est un nombre rond ». Si tu declares `Var moyenne : entier` et que le calcul donne 12,5, la partie décimale sera perdue : l'algorithme affichera 12 au lieu de 12,5, ce qui peut changer la décision d'admission d'un élève ! Règle pratique : dès qu'une valeur peut être décimale (prix, moyenne, taille, taux), déclare-la toujours en `reel`.

EXERCICE RÉSOLU 3 Déclarer les données d'une cantine scolaire

Énoncé : la cantine du collège vend des repas. On veut écrire un algorithme qui, à partir du prix hors taxes d'un repas, calcule le prix TTC (taux de TVA : 19,25 %) et indique si le repas est payé ou non. Écris la partie déclarative complète de cet algorithme en justifiant le type de chaque donnée.

SOLUTION

Solution détaillée : identifions d'abord toutes les données du problème.

- Le taux de TVA : il est fixé à 19,25 et ne change jamais pendant l'exécution → constante
TAUX = 19,25.
- Le prix hors taxes du repas : c'est un nombre qui peut être décimal (par exemple 500 FCFA ou 750,50 FCFA) et qui varie d'un repas à l'autre → variable de type `reel`.
- Le prix TTC : résultat d'un calcul avec un taux décimal, il est forcément susceptible d'être décimal → variable de type `reel`.
- L'information « le repas est-il payé ? » : la réponse est oui ou non → variable de type `booléen`.

La partie déclarative s'écrit donc :

```
Algorithme Cantine
Const TAUX = 19,25
Var prixHT, prixTTC : reel
Var estPaye : booléen
Debut
    (* instructions de calcul *)
Fin
```

On remarque que le taux est déclaré en constante (valeur fixe), que les deux prix sont déclarés en `reel` (et non en `entier`, pour ne pas perdre les centimes), et que l'état du paiement est un booléen puisqu'il ne vaut que Vrai ou Faux.

EXERCICE 3 À toi de jouer

Le secrétariat de ton collège veut un algorithme pour gérer la fiche d'un élève. Pour chacune des données suivantes, indique s'il s'agit d'une variable ou d'une constante, donne-lui un nom correct et précise son type en justifiant : le nom de l'élève ; sa date de naissance (année seulement) ; sa moyenne du trimestre ; le nombre de classes du collège (fixé à 12) ; l'information « l'élève a payé la scolarité » ; la première lettre de son quartier.

CORRIGÉ Exercice 1

- Nom de l'élève : variable `nom` de type `chaine` (texte de plusieurs caractères, différent pour chaque élève).
- Année de naissance : variable `anneeNaissance` de type `entier` (un nombre entier comme 2011).
- Moyenne du trimestre : variable `moyenne` de type `reel` (peut être décimale : 12,5).
- Nombre de classes : constante `NB_CLASSES = 12` (valeur fixe, non précisée car le type est déduit de la valeur).
- Scolarité payée : variable `scolaritePayee` de type `booléen` (Vrai ou Faux).
- Première lettre du quartier : variable `initiale` de type `caractere` (une seule lettre, par exemple 'B' pour Bonabéri).

À RETENIR L'essentiel de la section

- Une variable est un emplacement mémoire nommé dont la valeur peut changer pendant l'exécution ; une constante est un emplacement nommé dont la valeur reste fixe.
- On déclare une variable par `Var nom : type` et une constante par `Const NOM = valeur`, dans la partie déclarative, avant `Debut`.
- Les cinq types élémentaires sont : `entier`, `reel`, `caractere` (entre apostrophes), `chaine` (entre guillemets) et `booléen` (Vrai/Faux).
- Pour choisir un type, on se demande quelles valeurs la donnée peut prendre, en pensant aux cas limites : toute valeur susceptible d'être décimale (prix, moyenne) se déclare en `reel`.

Maintenant que nous savons réserver et nommer nos « boîtes mémoire » avec le bon type, il reste à apprendre à les utiliser : comment mettre une valeur dans une variable, comment demander une information à l'utilisateur et comment afficher un résultat. C'est précisément l'objet de la section suivante, consacrée aux instructions de base : la lecture, l'écriture et l'affectation.

4 STRUCTURE DE BASE D'UN ALGORITHME

4.1 Variables, constantes et types de données

Avant d'écrire le moindre calcul, un algorithme doit nommer et typer les données qu'il va manipuler. C'est le rôle de la partie **déclarative** (avant le `Début`).

DÉFINITION Variable

Une **variable** est une zone mémoire nommée dont la valeur peut changer au cours de l'exécution de l'algorithme. Elle est caractérisée par :

- un nom (identificateur) : commence par une lettre, sans espace, sans accent (ex. `total`, `nbBeignets`, `moyenneClasse`);
- un type : l'ensemble des valeurs qu'elle peut contenir;
- une valeur actuelle : le contenu de la case mémoire à un instant donné.

DÉFINITION Constante

Une **constante** est une zone mémoire nommée dont la valeur ne change jamais pendant l'exécution. Elle est fixée dès la déclaration.

Constante <nom> = <valeur> : <Type>

Exemple : Constante `PRIX_BEIGNET = 100 : Entier`

PROPRIÉTÉ Types de données de base (niveau 3ème)

Type	Ensemble des valeurs	Exemples littéraux
Entier	$\mathbb{Z} = \{\dots, -2, -1, 0, 1, 2, \dots\}$	5, -3, 0
Réel	$\mathbb{R}$ (nombres à virgule)	3,14, -0,5, 12,0
Caractère	Un seul symbole (lettre, chiffre, signe)	'A', '7', '+'
Booléen	{Vrai, Faux}	Vrai, Faux

Note : En algorithmique au BEPC, on écrit les réels avec une virgule (notation française), pas un point.

PROPRIÉTÉ Syntaxe de la déclaration

- **Variables :** Variable <liste noms> : <Type> (séparés par des virgules).
- **Constantes :** Constante <nom> = <valeur> : <Type> (une par ligne).

La zone de déclaration se place avant le mot-clé `Début`.

EXEMPLE Déclarations variées

```
Algorithme ExempleDeclarations
Constante TVA = 19,25 : Reel
Constante NOM_ETAB = "Lycee Bilingue" : Chaîne
Variable age, note : Entier
Variable moyenne, prix : Reel
Variable reponse : Caractere
Variable admis : Booleen
Debut
    // ... corps de l'algorithme
Fin
```

Remarque : Le type `Chaîne` (chaîne de caractères) est souvent accepté pour les messages longs, bien que le type de base soit `Caractere`.

ATTENTION Règles de nommage (identificateurs)

- Commencer par une lettre (pas de chiffre, pas de `_` au début).
- Pas d'espace, pas d'accent (`moyenne_classe autorisé`, `moyenne classe interdit`, `âge interdit`).
- Respecter la casse : `total` et `Total` sont deux variables différentes (sensible à la casse dans la plupart des langages, et recommandé en pseudo-code).
- Éviter les mots réservés du langage (`Si`, `Alors`, `Variable`, `Entier`, ...).

4.2 Les instructions de base : lecture, écriture, affectation

Maintenant que nos données sont déclarées et typées, il faut les faire vivre. Trois instructions fondamentales permettent l'interaction et le calcul : **Lire** (entrée), **Écrire** (sortie) et l'**affectation** (calcul/mémorisation).

Prenons une situation concrète : la cantine du collège vend le beignet à 100 FCFA. On veut un algorithme qui demande le nombre de beignets achetés, calcule le prix total, puis l'affiche. Il faut donc : **lire** une donnée, **calculer** (affecter), puis **écrire** le résultat.

4.3 L'instruction d'entrée : la lecture

4.4 L'instruction de sortie : l'écriture

Commençons par les deux instructions qui permettent à l'algorithme de communiquer avec l'utilisateur.

DÉFINITION Lecture et écriture

- L'instruction **Lire** (ou instruction d'entrée) permet à l'utilisateur de fournir une valeur au clavier; cette valeur est rangée dans une variable préalablement déclarée et de type compatible.
- L'instruction **Écrire** (ou instruction de sortie) permet d'afficher à l'écran un message (texte entre guillemets doubles) ou la valeur d'une variable / d'une expression. On peut enchaîner plusieurs arguments séparés par des virgules.

PROPRIÉTÉ Syntaxe exacte

- `Lire(variable)` : l'algorithme s'arrête, attend que l'utilisateur tape une valeur valide pour le type de la variable, puis la range dedans.
- `Écrire("message")` : affiche le texte tel quel (sans les guillemets).
- `Écrire(variable)` ou `Écrire(expression)` : affiche la valeur de la variable ou le résultat du calcul.
- **Combinaison** : `Écrire("Le total est : ", total, " FCFA")`.

EXEMPLE Premier algorithme complet : la cantine

```
Algorithme Cantine
Constante PRIX_UNITAIRE = 100 : Entier
Variable nb, total : Entier
Debut
    Écrire("Combien de beignets ? ")
    Lire(nb)                // l'utilisateur tape par exemple 5
    total <- nb * PRIX_UNITAIRE // calcul du prix total
    Écrire("Le prix total est : ", total, " FCFA")
Fin
```

Si l'utilisateur tape 5, l'écran affiche : Le prix total est : 500 FCFA.

ATTENTION Guillemets ou pas guillemets ?

`Écrire("total")` affiche le mot total, alors que `Écrire(total)` affiche la valeur de la variable (par exemple 500). C'est une erreur très fréquente au BEPC : le texte entre guillemets est affiché tel quel, sans guillemets c'est le contenu de la variable qui est affiché.

4.5 L'instruction d'affectation

Voici l'instruction la plus importante de toute l'algorithmique : c'est elle qui permet de calculer et de mémoriser les résultats.

DÉFINITION Affectation

L'**affectation** (symbole $\leftarrow$, flèche vers la gauche, saisi souvent $\leftarrow$ au clavier) consiste à donner à une variable la valeur d'une expression. Sa syntaxe est :

`variable  $\leftarrow$  expression`

On lit : « la variable reçoit la valeur de l'expression » ou « la variable prend la valeur de l'expression ».

PROPRIÉTÉ Sens de l'affectation – Règle d'or

L'affectation s'exécute toujours en deux étapes, dans cet ordre strict :

1. On évalue d'abord l'expression à droite de la flèche (on effectue le calcul avec les valeurs actuelles des variables) ;
2. Puis on range le résultat dans la variable à gauche, en écrasant son ancienne valeur. La variable de gauche doit être de type compatible avec le résultat de l'expression.

EXEMPLE L'ancienne valeur est écrasée

```
a  $\leftarrow$  5           // a contient 5
a  $\leftarrow$  a + 3      // 1. on évalue a + 3 = 5 + 3 = 8
                   // 2. on range 8 dans a (l'ancien 5 est perdu)
Ecrire(a)          // affiche 8
```

Après ces deux affectations, la variable `a` contient 8. L'instruction `a  $\leftarrow$  a + 3` se lit : « le nouveau `a` vaut l'ancien `a` plus 3 ».

ATTENTION Deux erreurs classiques à ne jamais commettre

- Sens interdit (côté gauche) : écrire `a + b  $\leftarrow$  s` est faux. À gauche de la flèche, il doit y avoir une seule variable (un nom de case mémoire), jamais un calcul ni une constante. C'est la variable qui reçoit, pas l'expression !
- Confusion avec les mathématiques : en maths, $a = a + 3$ est une équation impossible (aucun nombre n'est égal à lui-même plus 3). En algorithmique, `a  $\leftarrow$  a + 3` est parfait.

tement correct : ce n'est pas une égalité, c'est un transfert d'une nouvelle valeur. Le symbole $\leftarrow$ n'a rien à voir avec le signe $=$ des mathématiques.

4.6 Les opérateurs

Les expressions à droite de la flèche (ou dans les conditions, ou dans Écrire) sont construites avec des **opérateurs**.

PROPRIÉTÉ Opérateurs arithmétiques (sur Entiers et Réels)

Opérateur	Signification	Exemple (résultat)
+	addition	7 + 2 donne 9
-	soustraction	7 - 2 donne 5
*	multiplication	7 * 2 donne 14
/	division réelle (résultat Réel)	7 / 2 donne 3,5
div	quotient de la division euclidienne (Entiers)	7 div 2 donne 3
mod	reste de la division euclidienne (Entiers)	7 mod 2 donne 1

Priorités : *, / , div , mod prioritaires sur + , - . Parenthèses () pour forcer l'ordre.

PROPRIÉTÉ Opérateurs de comparaison (résultat : Booléen)

Ils comparent deux valeurs de même type (Entier, Réel, Caractère) et renvoient Vrai ou Faux.

Symbole	Sens
=	égal à
≠ (<> parfois toléré)	différent de
<	strictement inférieur à
>	strictement supérieur à
≤	inférieur ou égal à
≥	supérieur ou égal à

Exemple : `age >= 15` vaut Vrai si `age` contient 17.

EXEMPLE Bons de cantine : div et mod en action

Un élève a 750 FCFA. Combien de beignets à 100 FCFA peut-il acheter, et combien lui restera-t-il ?

```
nb <- 750 div 100 // nb contient 7 (quotient entier)
reste <- 750 mod 100 // reste contient 50 (reste entier)
Ecrire("Nombre : ", nb, " Reste : ", reste)
```

Il achète 7 beignets et il lui reste 50 FCFA. Vérification : $7 \times 100 + 50 = 750$.

4.7 Rédaction complète d'algorithmes et vérification par trace

Comment être sûr qu'un algorithme est correct avant de le faire exécuter par un ordinateur (ou de le rendre au BEPC)? En jouant soi-même le rôle de la machine, à la main : c'est la **trace d'exécution** (ou **tableau de trace**).

MÉTHODE Construire un tableau de trace

Pour vérifier un algorithme, on construit un tableau de trace :

1. On crée une colonne pour le numéro/texte de l'instruction et une colonne par variable de l'algorithme (dans l'ordre de déclaration) ;
2. On exécute les instructions dans l'ordre séquentiel, une ligne par instruction exécutée ;
3. Après chaque instruction, on note la nouvelle valeur de chaque variable modifiée (une variable non modifiée garde sa valeur : on recopie ou on laisse vide) ;
4. À la fin, on vérifie que les valeurs obtenues correspondent au résultat attendu (calcul mental, logique métier).

4.8 Exemple guidé : échanger le contenu de deux variables

Problème. Deux variables *a* et *b* contiennent respectivement 5 et 9. On veut échanger leurs contenus : à la fin, *a* doit contenir 9 et *b* doit contenir 5. C'est comme échanger le contenu de deux verres pleins : il faut un troisième verre vide !

ATTENTION Pourquoi pas simplement *a* <- *b* puis *b* <- *a* ?

Si l'on écrit *a* <- *b* d'abord, la valeur 9 écrase le 5 contenu dans *a* : le 5 est perdu à jamais. Ensuite *b* <- *a* recopie 9 dans *b*. Résultat : les deux variables contiennent 9 ! Échanger deux variables sans variable intermédiaire écrase l'une des valeurs.

MÉTHODE L'échange avec une variable temporaire

1. Sauvegarder la valeur de `a` dans une variable `temp` (ou `aux`, `tampon`);
2. Recopier la valeur de `b` dans `a`;
3. Recopier la valeur sauvegardée (`temp`) dans `b`.

EXEMPLE Algorithme d'échange complet

```

Algorithme Echange
Variable a, b, temp : Entier
Debut
  a <- 5
  b <- 9
  temp <- a  // 1. Sauvegarde : temp vaut 5
  a <- b     // 2. a reçoit 9
  b <- temp  // 3. b reçoit l'ancien a (5)
  Ecrire("Après échange : a = ", a, " b = ", b)
Fin

```

Instruction exécutée	a	b	temp
Début (init.)	?	?	?
a <- 5	5	?	?
b <- 9	5	9	?
temp <- a	5	9	5
a <- b	9	9	5
b <- temp	9	5	5
Ecrire(...)	9	5	5
Fin	9	5	5

Tableau de trace de l'algorithme d'échange. Les valeurs modifiées sont surlignées (vert = initialisation/sauvegarde, orange = modification cible). À la fin, `a` et `b` ont bien échangé leurs contenus grâce à `temp`.

EXPÉRIENCE TP guidé : Trace manuelle d'un calcul de moyenne

Objectif : S'entraîner à construire une trace complète sur papier. Matériel : Feuille, stylo.

Algorithme à tracer :

```
Algorithme MoyenneDeuxNotes
Variable n1, n2, somme, moy : Reel
Debut
    Ecrire("Note 1 : "); Lire(n1)
    Ecrire("Note 2 : "); Lire(n2)
    somme <- n1 + n2
    moy <- somme / 2
    Ecrire("Moyenne = ", moy)
Fin
```

Manipulation :

1. Dessiner le tableau (colonnes : Instruction, n1, n2, somme, moy).
2. Simuler l'exécution pour $n1 = 12$ et $n2 = 16$.
3. Remplir le tableau ligne par ligne.
4. Vérifier que moy vaut bien 14 à la fin.

Interprétation : La trace prouve que l'algorithme calcule correctement la moyenne arithmétique. C'est la méthode de validation officielle au BEPC.

EXERCICE RÉSOLU 4 Prix de trois cahiers et monnaie

Énoncé. Un cahier coûte 350 FCFA. Écrire un algorithme qui demande le nombre de cahiers achetés, calcule le prix total, puis affiche le prix total et la monnaie à rendre si l'élève paie avec un billet de 2000 FCFA. Donner la trace pour 4 cahiers.

SOLUTION**Solution détaillée.**

```
Algorithme AchatCahiers
Constante PRIX_CAHIER = 350 : Entier
Constante BILLET = 2000 : Entier
Variable nb, total, monnaie : Entier
Debut
    Ecrire("Nombre de cahiers ? ")
    Lire(nb) // L'utilisateur tape 4
    total <- nb * PRIX_CAHIER // total = 4 * 350 = 1400
    monnaie <- BILLET - total // monnaie = 2000 - 1400 = 600
    Ecrire("Total à payer : ", total, " FCFA")
    Ecrire("Monnaie à rendre : ", monnaie, " FCFA")
Fin
```

Trace d'exécution (pour nb = 4) :

Instruction	nb	total	monnaie
Début	?	?	?
Lire(nb)	4	?	?
total <- nb * 350	4	1400	?
monnaie <- 2000 - total	4	1400	600
Ecrire(...)	4	1400	600

L'écran affiche : Total a payer : 1400 FCFA puis Monnaie a rendre : 600 FCFA. Le résultat est correct : $4 \times 350 = 1400$ et $2000 - 1400 = 600$.

LESAVAIS-TU ? Le mot « algorithme » et Al-Khwarizmi

Le mot algorithme vient du nom du mathématicien perse Al-Khwarizmi (vers 780–850), qui a travaillé à la « Maison de la Sagesse » de Bagdad. Son livre Al-Kitab al-mukhtasar fi hisab al-jabr wa'l-muqabala a donné le mot « algèbre » et a systématisé les méthodes de calcul étape par étape (les algorithmes) pour résoudre les équations. Aujourd'hui, chaque fois que vous suivez une recette de cuisine, un mode d'emploi de montage de meuble, ou que vous faites une division posée, vous exécutez un algorithme !

À RETENIR L'essentiel de la leçon – Structure de base

- **Déclaration** : Variable <noms> : <Type> | Constante <nom> = <val> : <Type> (avant Début).
- **Types de base** : Entier, Reel, Caractere, Booléen.
- Lire(variable) : entrée clavier → variable.
- Écrire(...) : sortie écran (texte "..." ou valeurs).
- variable <- expression : 1. Calculer à droite, 2. Ranger à gauche (écrase l'ancien).
- **Opérateurs** : + - * / div mod (arithmétiques); = < > <= >= (comparatifs, renvoient Booléen).
- Trace d'exécution : tableau (Instruction × Variables) pour valider la logique pas à pas.
- Échange de deux variables : obligatoirement via une variable temporaire (temp).

EXERCICE 4 À toi de jouer – Entraînement BEPC

1. Calcul successif. Que contient la variable x après la séquence : $x \leftarrow 10$; $x \leftarrow x - 4$; $x \leftarrow x * 2$?
2. Algorithme complet. Écrire un algorithme qui lit le prix unitaire d'un stylo (entier) et la quantité achetée (entier), puis affiche le montant total à payer (avec libellé « Montant : ... FCFA »).
3. Trace table. Construire le tableau de trace complet de l'algorithme suivant pour les valeurs initiales $a = 12$ et $b = 5$:

```
q ← a div b
r ← a mod b
Ecrire(q, r)
```

4. Erreur ou pas? Dire si les instructions suivantes sont syntaxiquement correctes (selon nos conventions). Si non, corriger.
 - $5 \leftarrow x$
 - $total \leftarrow "100"$
 - Lire("Entrez age : ", age)
 - Ecrire(age $\geq$ 18)

CORRIGÉ Exercices – Corrigé détaillé

1. $x \leftarrow 10 \Rightarrow x = 10$. $x \leftarrow x - 4 \Rightarrow$ évalue $10 - 4 = 6$, range 6 dans x . $x \leftarrow x * 2 \Rightarrow$ évalue $6 * 2 = 12$, range 12 dans x . Réponse finale : 12.

```
Algorithme AchatStylos
Variable prix, qte, montant : Entier
Debut
  Ecrire("Prix unitaire du stylo ? "); Lire(prix)
  Ecrire("Quantité ? "); Lire(qte)
  montant ← prix * qte
  Ecrire("Montant : ", montant, " FCFA")
Fin
```

3. Trace :

Instruction	q	r
Début	?	?
$q \leftarrow a \text{ div } b$ (12 div 5)	2	?
$r \leftarrow a \text{ mod } b$ (12 mod 5)	2	2
Ecrire(q, r)	2	2

L'écran affiche 2 2 (car $12 = 5 \times 2 + 2$).

4. Analyse :

- `5 <- x` : INCORRECT. Côté gauche = littéral (constante). Correction : `x <- 5`.
- `total <- "100"` : INCORRECT si `total` est Entier (incompatibilité type). Correction : `total <- 100` ou déclarer `total` : Chaîne.
- `Lire("Entrez age : ", age)` : INCORRECT. Lire ne prend qu'un seul argument (la variable). Correction : `Ecrire("Entrez age : "); Lire(age)`.
- `Ecrire(age >= 18)` : CORRECT. Affiche Vrai ou Faux selon la valeur de `age`.

5 RAPPELS ET NOTION DE LANGAGE DE DÉFINITION DES ALGORITHMES

Avant d'écrire notre première ligne d'algorithme, posons-nous la question fondamentale : qu'est-ce qu'un algorithme ? Imagine que tu donnes la recette du ndolé à un ami : « Lave les feuilles, fais-les bouillir, ajoute l'arachide, le poisson, laisse mijoter... ». Cette suite d'étapes ordonnées, précises et finies pour obtenir un résultat garanti, **c'est un algorithme**. En informatique, l'ordinateur est ton « cuisinier » : il ne comprend que des ordres extrêmement précis, sans ambiguïté, exprimés dans un langage qu'il sait exécuter.

DÉFINITION Algorithme

Un **algorithme** est une suite finie, précise et ordonnée d'instructions élémentaires permettant de résoudre un problème ou d'obtenir un résultat à partir de données d'entrée. Il doit être général (fonctionner pour toute donnée valide) et efficace (terminer en un temps raisonnable).

PROPRIÉTÉ Caractéristiques d'un algorithme

Tout algorithme valide respecte quatre propriétés essentielles :

1. **Finitude** : Il se termine après un nombre fini d'étapes (pas de boucle infinie).
2. **Précision (non-ambiguïté)** : Chaque instruction a un sens unique, sans place à l'interprétation.
3. **Généralité** : Il traite une classe de problèmes (ex. : « calculer la moyenne de deux notes » fonctionne pour 10 et 12, comme pour 14,5 et 8,25).
4. **Entrées/Sorties définies** : Les données initiales (entrées) et les résultats attendus (sorties) sont explicitement identifiés.

5.1 Pourquoi un langage de définition (pseudo-code) ?

Tu connais le français (langage naturel) et tu entendras parler de Python, C, JavaScript (langages de programmation). Le langage naturel est trop flou : « ajoute un peu de sel » — combien ? Le langage machine (binaire) ou même le C sont trop rigides pour réfléchir à la logique. Le **pseudo-code** (ou langage algorithmique) est le langage de conception : il utilise des mots-clés en français (ou anglais) structurés comme du code, mais sans la syntaxe stricte d'un compilateur. C'est le « brouillon propre » du programmeur.

EXEMPLE Même problème, trois langages

Problème : Demander un nombre et afficher son double.

Français (naturel)	« Écris "Donne un nombre", lis le nombre, calcule le double, écris le résultat. »
Pseudo-code (cours 3ème)	<pre>Algorithme Double Var x, d : entier Debut Ecrire("Donne un nombre : ") Lire(x) d <- x * 2 Ecrire("Le double est ", d) Fin</pre>
Python (exécution)	<pre>x = int(input("Donne un nombre : ")) d = x * 2 print("Le double est", d)</pre>

Le pseudo-code est le pont entre la logique humaine et le code machine. Il est universel : on le comprend sans connaître Python ni C.

À RETENIR Le trio gagnant

- Langage naturel : pour formuler le besoin (cahier des charges).
- Pseudo-code : pour concevoir la logique (algorithme), indépendamment de la machine.
- Langage de programmation (C, Python...) : pour implémenter et exécuter sur l'ordinateur.

En 3ème, nous maîtrisons le pseudo-code : c'est la compétence examinée au BEPC.

6 STRUCTURE DE BASE D'UN ALGORITHME

Tout algorithme écrit en pseudo-code dans ce cours respecte une **architecture rigide en trois blocs**. Respecter cette structure, c'est garantir la lisibilité et la correction.

PROPRIÉTÉ Squelette standard d'un algorithme

```
Algorithme <NomSignificatif> // 1. En-tête : nom unique, sans espace, explicite
Const // 2. Bloc constantes (optionnel)
  <NOM_CONST> = <valeur> : <type> // Noms en MAJUSCULES par convention
Var // 3. Bloc variables (obligatoire si données)
  <nom_var> : <type> // Noms en minuscules (camelCase autorisé)
  <nom_var2>, <nom_var3> : <type> // Plusieurs variables de même type sur une ligne
Debut // 4. Début du corps principal
  <Instruction_1> // Instructions indentées (2 espaces)
  <Instruction_2>
  ...
Fin // 5. Fin obligatoire
```

MÉTHODE Règles de rédaction du pseudo-code

Pour que ton algorithme soit lu et corrigé facilement (par ton prof, par toi-même plus tard), applique ces règles :

1. **Nommage** : Algorithme CalculMoyenne (PascalCase), Const TVA = 19,25 (MAJUSCULES), Var note, moyenne : reel (minuscules).
2. **Indentation** : Décale de 2 espaces (ou une tabulation) tout ce qui est entre Debut et Fin. C'est visuel : on voit la structure.
3. **Commentaires** : Utilise // pour expliquer pourquoi, pas quoi (le code montre le quoi).
Ex : `m <- (n1+n2)/2 // moyenne arithmétique.`
4. **Une instruction par ligne** : Ne mets pas Lire(x) Ecrire(x) sur la même ligne.
5. **Fin de ligne** : En pseudo-code, pas de point-virgule ; à la fin des lignes (contrairement au C).

ATTENTION Pièges de structure fréquents

- Oublier Debut ou Fin : l'algorithme est invalide.
- Mélanger déclarations et instructions : Var et Const uniquement avant Debut.
- Écrire du code en dehors de Debut ... Fin : tout traitement doit être à l'intérieur.

- Mettre des accents dans les noms de variables : `var âge` → interdit. Utilise `age` ou `ageEleve`.

7 VARIABLES, CONSTANTES ET TYPES DE DONNÉES

Un algorithme manipule des valeurs. Pour les stocker en mémoire le temps du calcul, on utilise des **variables** (contenu changeant) ou des **constantes** (contenu figé). Chaque « boîte » a un **type** qui détermine quelles valeurs elle peut contenir et quelles opérations sont autorisées.

DÉFINITION Variable

Une **variable** est une zone de mémoire identifiée par un nom (identificateur), qui contient une seule valeur à la fois d'un type donné, et dont la valeur peut être modifiée pendant l'exécution par une affectation ou une lecture.

DÉFINITION Constante

Une **constante** est une zone de mémoire identifiée par un nom, qui contient une valeur d'un type donné, fixée à la déclaration et interdite de modification pendant toute l'exécution. Elle sert à nommer une valeur fixe (taux, seuil, π) pour la lisibilité et la maintenance.

PROPRIÉTÉ Types de données de base (niveau 3ème)

Type	Mot-clé pseudo-code	Valeurs admises	Exemples
Entier	entier	Nombres sans virgule (positifs, négatifs, zéro)	-5, 0, 42, 10000
Réel	reel	Nombres à virgule (notation décimale)	3,14 ; -12,5 ; 0,0 ; 19,25
Chaîne de caractères	chaine	Suite de caractères entre guillemets	"Yaoundé" ; "Note : " ; "123"
Booléen	booléen	Deux valeurs logiques seulement	Vrai, Faux

EXEMPLE Déclarations variées

```

Algorithme ExempleTypes
Const PI = 3,14159      // reel (implicite par la valeur)
Const NOM_ETAB = "Lycée Bilingue" // chaîne
Var age, nbEleves : entier
Var moyenne, taille : reel
Var nomEleve, mention : chaîne
Var estAdmis : booléen
Debut
    // ... corps de l'algorithme
Fin

```

ATTENTION Règles d'identificateurs (noms de variables/constants)

1. Commence par une lettre (a-z, A-Z). Pas de chiffre au début.
2. Autorisé : lettres, chiffres, _ (underscore). Interdit : espaces, accents (é, à, ç), caractères spéciaux (@, #, \$), mots-clés réservés (Var, Const, Si, Pour, Lire...).
3. La casse compte : `moyenne` et `Moyenne` sont deux variables différentes. Convention : variables en minuscules (ou camelCase), constantes en MAJUSCULES.
4. Choisis des noms significatifs : `mt` → mauvais ; `montantTTC` → excellent.

À RETENIR Variable vs Constante : le bon réflexe

- Constante : « Le prix du repas est 500 FCFA », « La TVA est 19,25% », « $\pi \approx 3,14$ ». → `Const PRIX = 500.`
- Variable : « Saisir la note de l'élève », « Calculer la moyenne », « Compter les élèves présents ». → `Var note, moyenne, compteur : ....`

Utiliser une constante pour une valeur fixe évite les « nombres magiques » dans le code et permet de changer la valeur en un seul endroit si la loi change (ex. TVA passant à 20%).

8 LES INSTRUCTIONS DE BASE : LECTURE, ÉCRITURE, AFFECTATION

Maintenant que nous savons nommer et typer nos données, voyons les trois instructions fondamentales qui font vivre l'algorithme : entrer des données, les transformer, sortir des résultats.

8.1 L'instruction de lecture : Lire

PROPRIÉTÉ Syntaxe Lire

`Lire(<variable>)` ou `Lire(<var1>, <var2>, ...)`

- Effectue une pause : l'algorithme attend que l'utilisateur tape une valeur au clavier et valide (Entrée).
- La valeur saisie est convertie selon le type de la variable (entier → entier, réel → réel, etc.) et stockée dans la variable.
- La variable doit être déclarée avant dans `Var`.
- Message utilisateur : `Lire` n'affiche rien. On place toujours un `Ecrire` avant pour guider l'utilisateur.

EXEMPLE Utilisation de Lire

```
Ecrire("Entrez votre age : ")
Lire(age)           // age est de type entier
Ecrire("Entrez votre taille (m) : ")
Lire(taille)        // taille est de type reel
Ecrire("Entrez votre nom : ")
```

```
| Lire(nom)           // nom est de type chaîne
```

8.2 L'instruction d'écriture : Écrire

PROPRIÉTÉ Syntaxe Écrire

Écrire(<expression_1>, <expression_2>, ...)

- Affiche à l'écran la suite des expressions, séparées par des virgules dans l'instruction.
- Les chaînes de caractères (messages) sont entre guillemets doubles "...".
- Les variables s'écrivent sans guillemets : leur valeur est affichée.
- On peut mélanger texte et variables : Écrire("La moyenne est ", m, " sur 20").
- Pas de retour à la ligne automatique standard (comportement dépend de l'interpréteur) ; on gère l'affichage par des Écrire successives.

ATTENTION Guillemets : le piège classique

```
// ERREUR : affiche littéralement "moyenne"  
Ecrire("moyenne")  
  
// CORRECT : affiche la VALEUR de la variable moyenne  
Ecrire(moyenne)  
  
// CORRECT : message + valeur  
Ecrire("Moyenne = ", moyenne)
```

Règle d'or : "texte" = texte fixe ; variable = valeur variable.

8.3 L'instruction d'affectation : ←

C'est le cœur du traitement. Elle calcule une expression et stocke le résultat dans une variable.

PROPRIÉTÉ Syntaxe et sémantique de l'affectation

<variable> ← <expression>

1. Évaluer l'expression à droite (calculer sa valeur actuelle en utilisant les valeurs actuelles des variables).
2. Stocker ce résultat dans la variable à gauche (écraser l'ancienne valeur).

3. Le type de l'expression doit être compatible avec le type de la variable (entier dans entier, réel dans réel, etc.).

Attention : $\leftarrow$ n'est PAS une égalité mathématique. $x \leftarrow x + 1$ signifie « prend la valeur actuelle de x, ajoute 1, remets dans x » (incrément). En maths, $x = x + 1$ est faux.

PROPRIÉTÉ Opérateurs arithmétiques et priorités

Opérateur	Sens	Priorité
+ , -	Addition, Soustraction	1 (la plus faible)
* , /	Multiplication, Division	2
^ ou **	Puissance	3 (la plus forte)
()	Parenthèses (forcent l'ordre)	Priorité absolue

Associativité : Pour une même priorité, on évalue de gauche à droite. Ex : $10 - 3 - 2 = (10-3)-2 = 5$.

EXEMPLE Affectations et priorités

```

Var a, b, c, res : reel
Debut
  a <- 10
  b <- 3
  c <- 2
  res <- a + b * c      // 1. b*c = 6, 2. a+6 = 16 (res = 16)
  res <- (a + b) * c    // 1. (a+b) = 13, 2. 13*2 = 26 (res = 26)
  res <- a / b * c      // 1. a/b = 3.333..., 2. *2 = 6.666... (gauche à droite)
  a <- a + 1           // Incrément : a devient 11
Fin
  
```

EXPÉRIENCE TP Papier : Trace d'exécution manuelle

Objectif : Comprendre l'état mémoire pas à pas. Matériel : Feuille, stylo. Algorithme :

```

Algorithme TraceDemo
Var x, y, z : entier
Debut
  x <- 5
  y <- 2
  z <- x + y
  
```

```

x ← -z * 2
y ← -x - y
Ecrire(x, y, z)
Fin

```

Manipulation : Complète le tableau de trace ci-dessous (colonnes = variables, lignes = instructions).

Instruction	x	y	z
$x \leftarrow 5$			
$y \leftarrow 2$			
$z \leftarrow x + y$			
$x \leftarrow z * 2$			
$y \leftarrow x - y$			
Écrire(x, y, z)	Affiche : ...		

Interprétation : Vérifie que la dernière ligne affiche 14 12 7. Si ce n'est pas le cas, reprends chaque ligne : l'affectation change une seule variable à la fois.

9 RÉDACTION COMPLÈTE D'ALGORITHMES SUR DES SITUATIONS CONCRÈTES

Dans les sections précédentes, tu as découvert les trois instructions de base du langage algorithmique : la **lecture** (Lire), l'**écriture** (Écrire) et l'**affectation** ($\leftarrow$). Chacune prise isolément est simple. Mais un algorithme utile n'est jamais une seule instruction : c'est une suite ordonnée d'instructions qui résout un problème concret, du début à la fin. Le moment est venu d'assembler ces briques pour rédiger des algorithmes complets, comme le ferait un programmeur face à un vrai besoin : calculer une moyenne de bulletin, établir une facture à la boutique, mesurer un champ. C'est toute la puissance de l'algorithmique : passer d'une situation de la vie courante à une méthode de calcul automatisable.

9.1 La démarche de résolution en cinq étapes

Face à un problème, le débutant a souvent envie d'écrire immédiatement des instructions. C'est une erreur : on construit un algorithme comme on construit une maison, en suivant un plan. Avant toute rédaction, il faut comprendre le problème et le découper.

MÉTHODE

Résoudre un problème par un algorithme : les 5 étapes

Pour passer d'une situation concrète à un algorithme complet, on suit toujours la même démarche :

1. Analyser le problème : lire attentivement l'énoncé, identifier les **données** (ce que l'utilisateur fournit, les entrées) et les **résultats** attendus (ce que l'algorithme doit produire, les sorties). Expliciter la ou les formules de calcul.
2. Déclarer : choisir un nom significatif et un type adapté pour chaque variable et chaque constante nécessaire (données d'entrée, résultats de sortie, valeurs fixes du problème).
3. Saisir : écrire les instructions Lire qui permettent d'entrer les données, précédées chacune d'un Écrire explicite (message de prompt).
4. Traiter : écrire les calculs (affectations) qui transforment les données en résultats, en utilisant les formules dégagées à l'étape 1. Respecter les priorités opératoires (parentheses!).
5. Afficher : écrire les instructions Écrire qui présentent les résultats à l'utilisateur, avec un message clair et l'unité si pertinent.

Ensuite, on vérifie l'algorithme par une **trace d'exécution** : on choisit des valeurs concrètes, on « joue » l'algorithme à la main et on compare le résultat obtenu au calcul manuel.

DÉFINITION Trace d'exécution

La **trace d'exécution** (ou test d'exécution) d'un algorithme est le tableau qui montre, ligne par ligne, la valeur prise par chaque variable au cours du déroulement, pour des données d'entrée choisies. Elle permet de vérifier que l'algorithme produit bien le résultat attendu, sans ordinateur.

9.2 Situation 1 : la moyenne trimestrielle d'un élève

Problème. Au collège, le professeur principal veut calculer rapidement la moyenne trimestrielle d'un élève en informatique, obtenue à partir de deux notes : la note du devoir et la note de la composition. La formule est :

$$m = \frac{n_1 + n_2}{2}$$

Analyse (étape 1).

- Entrées : deux notes n_1 et n_2 (des nombres pouvant avoir des virgules, comme 12,5). Type `reel`.
- Sortie : la moyenne m . Type `reel`.
- Traitement : additionner les deux notes, puis diviser par 2.

Déclaration (étape 2). Trois variables de type **reel** : n_1 , n_2 et m .

EXEMPLE Algorithme MoyenneTrimestre

```

Algorithme MoyenneTrimestre
Var n1, n2, m : reel      // les deux notes et la moyenne
Debut
  Ecrire("Entrez la premiere note : ")
  Lire(n1)                // saisie de la note 1
  Ecrire("Entrez la deuxieme note : ")
  Lire(n2)                // saisie de la note 2
  m <- (n1 + n2) / 2      // traitement : formule de la moyenne
  Ecrire("La moyenne est : ", m)
Fin

```

Vérification par une trace. Testons avec $n_1 = 12$ et $n_2 = 16$:

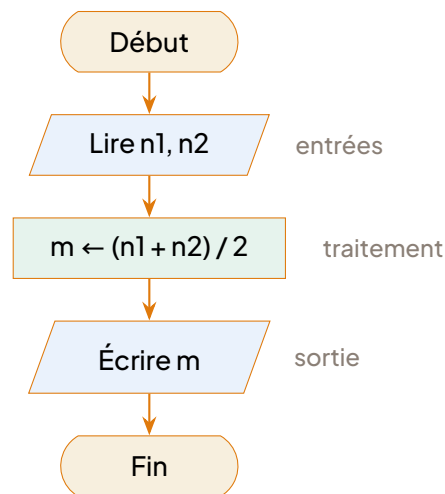
Instruction	n1	n2	m
Lire(n1)	12	?	?
Lire(n2)	12	16	?
$m \leftarrow (n1 + n2)/2$	12	16	14
Écrire(...)	affiche : « La moyenne est : 14 »		

Le calcul manuel donne $(12 + 16)/2 = 28/2 = 14$: l'algorithme est correct.

ATTENTION Le piège des parenthèses

L'erreur la plus fréquente ici est d'écrire $m \leftarrow n1 + n2 / 2$ sans parenthèses. Or la division est prioritaire sur l'addition : l'ordinateur calcule d'abord $n2 / 2$, puis ajoute $n1$. Avec $n_1 = 12$ et $n_2 = 16$, on obtiendrait $12 + 8 = 20$ au lieu de 14 ! La formule correcte exige les parenthèses : $(n1 + n2) / 2$. Retiens : en algorithmique comme en mathématiques, les priorités opératoires s'appliquent.

La figure suivante montre l'**organigramme** de cet algorithme : c'est une représentation graphique du déroulement, où l'on voit clairement la logique entrée → traitement → sortie, en parallèle du pseudo-code.



Organigramme de l'algorithme MoyenneTrimestre : chaque étape du pseudo-code correspond à un symbole (trapèze pour les entrées/sorties, rectangle pour le traitement).

9.3 Situation 2 : le montant TTC d'un achat à la boutique

Problème. La boutique du quartier affiche ses prix hors taxe (HT). La gérante veut un outil qui calcule le montant toutes taxes comprises (TTC) en appliquant la TVA camerounaise de 19,25 %. La formule est :

$$\text{TTC} = \text{HT} + \text{HT} \times \frac{19,25}{100}$$

Analyse.

- Entrée : le montant hors taxe ht (en FCFA). Type `reel`.
- Sortie : le montant TTC ttc . Type `reel`.
- Donnée fixe : le taux de TVA, 19,25, qui ne change jamais pendant l'exécution : c'est une **constante**.

C'est l'occasion de justifier un choix important : la TVA est déclarée comme constante car sa valeur est fixée par la loi et ne doit pas être modifiée par l'utilisateur ; les montants ht et ttc sont des variables réelles car ils changent à chaque client et peuvent comporter des décimales.

EXEMPLE Algorithme AchatTTC

```

Algorithme AchatTTC
Const TVA = 19,25          // taux de TVA en pourcentage (fixe)
Var ht, ttc : reel        // montant hors taxe et montant TTC
Debut
  Ecrire("Entrez le montant hors taxe (FCFA) : ")
  Lire(ht)                // saisie du prix HT
  ttc <- ht + ht * TVA / 100 // calcul du prix TTC

```

```

Ecrire("Le montant TTC est : ", ttc, " FCFA")
Fin

```

Vérification. Avec $ht = 10\,000$ FCFA :

$$TTC = 10\,000 + 10\,000 \times \frac{19,25}{100} = 10\,000 + 1\,925 = 11\,925 \text{ FCFA}$$

L'algorithme affiche bien 11925 FCFA : le test confirme le calcul manuel.

À RETENIR Variable ou constante ?

On déclare en **constante** toute valeur fixe du problème (un taux, un prix unitaire, le nombre π) : elle est définie une fois pour toutes et ne peut pas être modifiée pendant l'exécution. On déclare en **variable** tout ce qui est saisi par l'utilisateur ou calculé par le programme. Ce choix rend l'algorithme plus sûr et plus lisible.

9.4 Situation 3 : périmètre et aire d'un champ rectangulaire

Problème. Un agriculteur de Ngaoundéré possède un champ rectangulaire. Il veut connaître le **périmètre** (pour acheter la longueur de fil de clôture) et l'**aire** (pour estimer la quantité de semences). Pour un rectangle de longueur L et de largeur l :

$$P = 2 \times (L + l) \quad \text{et} \quad A = L \times l$$

Analyse.

- Entrées : la longueur L et la largeur l , en mètres (réels).
- Sorties : le périmètre P (en mètres) et l'aire A (en mètres carrés).
- Traitements : deux calculs indépendants, effectués l'un après l'autre.

EXEMPLE Algorithme ChampRectangulaire

```

Algorithme ChampRectangulaire
Var L, l, P, A : reel      // dimensions, perimetre et aire
Debut
  Ecrire("Longueur du champ (m) : ")
  Lire(L)
  Ecrire("Largeur du champ (m) : ")
  Lire(l)
  P <- 2 * (L + l)          // perimetre du rectangle
  A <- L * l                // aire du rectangle
  Ecrire("Le perimetre est : ", P, " m")
  Ecrire("L'aire est : ", A, " m2")
Fin

```

Vérification. Avec $L = 50$ m et $l = 30$ m : $P = 2 \times (50 + 30) = 160$ m de clôture, et $A = 50 \times 30 = 1500$ m². La trace d'exécution donne les mêmes valeurs.

9.5 Exercice résolu et synthèse

EXERCICE RÉSOLU 5 La cantine du lycée

La cantine d'un lycée de Yaoundé vend le repas à 500 FCFA. Le surveillant veut un algorithme qui demande le nombre d'élèves inscrits et affiche la recette totale attendue.

SOLUTION

Étape 1 — Analyse. Entrée : le nombre d'élèves n (un entier). Sortie : la recette r (en FCFA).
Donnée fixe : le prix du repas, 500 FCFA, déclaré en constante. Traitement : $r = n \times 500$.
Étapes 2 à 5 — Algorithme.

```

Algorithme RecetteCantine
Const PRIX = 500           // prix d'un repas en FCFA (fixe)
Var n, r : entier          // nombre d'eleves et recette
Debut
  Ecrire("Nombre d'eleves inscrits : ")
  Lire(n)
  r ← n * PRIX             // calcul de la recette
  Ecrire("La recette attendue est : ", r, " FCFA")
Fin
  
```

Vérification. Avec $n = 240$ élèves : $r = 240 \times 500 = 120\,000$ FCFA. Le calcul manuel confirme le résultat. La démarche est justifiée : n est un entier car on ne compte pas une fraction d'élève ; $PRIX$ est une constante car le tarif ne change pas pendant l'exécution.

LESAVAIS-TU ? Le pseudo-code est universel

Les algorithmes que tu viens d'écrire en pseudo-code français peuvent être traduits presque mot à mot dans n'importe quel langage de programmation : Python, C, Java, JavaScript... Seule la syntaxe change ; la logique (saisir, calculer, afficher) reste identique. C'est pourquoi les programmeurs du monde entier commencent toujours par réfléchir en algorithmes avant de coder : le pseudo-code est leur langue commune.

EXERCICE 5 À toi de jouer

1. Un taxi-moto (benskin) facture la course 100 FCFA de prise en charge plus 50 FCFA par kilomètre parcouru. Rédige l'algorithme complet (analyse, déclarations, instructions) qui calcule le prix d'une course, puis vérifie-le avec une distance de 8 km.
2. Rédige l'algorithme qui calcule la moyenne générale d'un élève à partir de trois notes,

et teste-le avec 10, 14 et 16.

CORRIGÉ Exercices 1 et 2

Exercice 1. Analyse. Entrée : distance d (reel); sortie : prix p (reel). Constantes : prise en charge 100, prix au km 50. Traitement : $p = 100 + 50 \times d$.

```

Algorithme PrixCourse
Const PRISE = 100
Const PRIXKM = 50
Var d, p : reel
Debut
  Ecrire("Distance parcourue (km) : ")
  Lire(d)
  p ← PRISE + PRIXKM * d
  Ecrire("Le prix de la course est : ", p, " FCFA")
Fin
  
```

Test avec $d = 8$: $p = 100 + 50 \times 8 = 500$ FCFA. Correct.

Exercice 2. Analyse. Entrées : trois notes $n1$, $n2$, $n3$ (reel). Sortie : moyenne m (reel). Traitement : $m = (n1 + n2 + n3)/3$.

```

Algorithme MoyenneGenerale
Var n1, n2, n3, m : reel
Debut
  Ecrire("Note 1 : "); Lire(n1)
  Ecrire("Note 2 : "); Lire(n2)
  Ecrire("Note 3 : "); Lire(n3)
  m ← (n1 + n2 + n3) / 3
  Ecrire("La moyenne generale est : ", m)
Fin
  
```

Test : $(10 + 14 + 16)/3 = 40/3 \approx 13,33$. Correct.

À RETENIR L'essentiel de la section

Tout algorithme de cette leçon suit le même squelette : déclarer les variables et constantes, saisir les données avec `Lire`, traiter par affectation, afficher les résultats avec `Écrire`. On vérifie toujours par une trace d'exécution comparée au calcul manuel. Ces algorithmes sont dits **séquentiels** : les instructions s'exécutent dans l'ordre, de haut en bas, sans choix ni répétition. Dans l'unité suivante, tu les enrichiras avec les structures conditionnelles (`Si... Alors`) et les boucles (`Pour`, `Tant que`), qui permettront de traiter des situations bien plus riches.

10 ACTIVITÉ D'INTÉGRATION

11 LEÇON 13 : PRÉSENTATION DU LANGAGE DE DÉFINITION DES ALGORITHMES

11.1 Objectifs de l'activité

À l'issue de cette activité d'intégration, tu seras capable de :

- analyser une situation concrète de la vie courante pour y repérer les **entrées**, les **traitements** et les **sorties** ;
- choisir correctement les **noms** et les **types** des variables et des constantes d'un algorithme ;
- rédiger un algorithme complet en **pseudo-code**, en respectant la structure de base (entête, déclarations, corps) ;
- vérifier le bon fonctionnement d'un algorithme à l'aide d'un **tableau de trace** ;
- présenter et **justifier** ta démarche devant la classe, en rédigeant une conclusion claire.

11.2 Situation

Au marché Mokolo, à Yaoundé, le jeune Armand tient un petit kiosque de vente de crédit téléphonique (MTN, Orange, Nexttel). Comme tous les vendeurs de son quartier, Armand n'achète pas le crédit au prix où il le revend : il l'achète à un **prix de gros** auprès d'un distributeur, puis il le revend à ses clients en ajoutant une **marge fixe de 5 %** sur le prix d'achat.

Chaque jour, Armand effectue des dizaines de ventes de montants différents : 500 FCFA, 1 000 FCFA, 2 500 FCFA, etc. À chaque fois, il doit calculer mentalement deux nombres :

- le **prix de vente** à demander au client : prix d'achat augmenté de 5 % ;
- son **bénéfice** sur la vente : la différence entre le prix de vente et le prix d'achat.

Un soir, en rentrant chez lui, Armand se trompe dans ses comptes et croit avoir gagné plus que la réalité. Il se dit alors : « Si seulement j'avais une petite procédure automatique qui, à partir du prix d'achat saisi, me donne immédiatement le prix de vente et mon bénéfice ! »

Armand sait que tu apprends l'algorithmique en classe de 3ème. Il te demande donc de l'aider. Pour tester ta solution, il te propose une vente réelle de sa journée : un achat de crédit dont le **prix d'achat est de 2 000 FCFA**.

Problème posé : rédiger un algorithme complet, en pseudo-code, qui lit le prix d'achat d'un crédit, calcule le prix de vente (marge de 5 %) et le bénéfice, puis affiche ces deux résultats. Vérifier ensuite cet algorithme par un tableau de trace avec un achat de 2 000 FCFA.

11.3 Consignes

Travail en groupes de 3 ou 4 élèves. Chaque groupe répond par écrit aux questions suivantes, puis un rapporteur présente la solution du groupe devant la classe.

1. **Analyse du problème.** Pour cette situation, identifiez et notez sur votre feuille :
 - (a) les **données d'entrée** (ce que l'utilisateur fournit à l'algorithme) ;
 - (b) les **résultats de sortie** (ce que l'algorithme doit afficher) ;
 - (c) les **traitements** (les calculs à effectuer pour passer des entrées aux sorties). Écrivez les formules mathématiques utilisées.
2. **Choix des variables et des constantes.** Proposez un nom significatif pour chaque information manipulée, en précisant s'il s'agit d'une **variable** ou d'une **constante**, ainsi que son **type** (entier, réel, chaîne de caractères...). Justifiez chaque choix : pourquoi la marge est-elle une constante et non une variable ?
3. **Rédaction de l'algorithme.** Rédigez l'algorithme complet en pseudo-code, en respectant les trois parties de la structure de base : l'**en-tête** (nom de l'algorithme), la partie **déclarations** (constantes et variables avec leurs types) et le **corps** (instructions entre Début et Fin). Utilisez les instructions de lecture, d'affectation et d'écriture vues dans la leçon.
4. **Vérification par une trace d'exécution.** Construisez le tableau de trace de votre algorithme pour un prix d'achat de 2 000 FCFA : un tableau dont les colonnes sont les noms des variables et dont chaque ligne correspond à une instruction exécutée. Vérifiez que les valeurs affichées à la fin sont bien celles attendues (faites le calcul à la main pour comparer).
5. **Présentation et justification.** Le rapporteur présente la solution du groupe au tableau et justifie chaque choix (noms, types, ordre des instructions). La classe valide la solution ou propose des corrections. Que se passerait-il si l'on calculait le bénéfice avant le prix de vente ?

11.4 Ressources à mobiliser

À RETENIR**Ce qu'il faut avoir en tête pour réussir**

- Structure de base d'un algorithme : un **en-tête** (Algorithme nom_de_l_algorithme), une partie **déclarations** (Constante, Variable avec le type de chacune), et un **corps** délimité par **Début** et **Fin**.
- Instructions de base : **Lire(variable)** pour saisir une entrée ; **variable ← expression** pour l'affectation ; **Écrire(...)** pour afficher un résultat ou un message.
- Variable et constante : une **variable** peut changer de valeur pendant l'exécution ; une **constante** garde toujours la même valeur, fixée dans les déclarations.
- Types de données : Entier, Réel, Caractère, Chaîne, Booléen. Un prix peut comporter des centimes : on choisit le type Réel.

- Calcul d'un pourcentage : augmenter une valeur de 5 % revient à la multiplier par $1 + \frac{5}{100} = 1,05$.
- Tableau de trace : tableau donnant, instruction après instruction, la valeur de chaque variable ; il permet de vérifier l'algorithme « à la main ».

11.5 Démarche et corrigé commenté

EXERCICE RÉSOLU 6 L'algorithme du vendeur de crédit téléphonique

Énoncé. Armand revend le crédit téléphonique avec une marge fixe de 5 % sur le prix d'achat. Rédiger un algorithme complet qui lit le prix d'achat d'un crédit, calcule puis affiche le prix de vente et le bénéfice. Vérifier l'algorithme par un tableau de trace pour un achat de 2 000 FCFA.

SOLUTION

Étape 1 : analyse du problème (entrées, traitements, sorties).

- Entrée : le prix d'achat du crédit, fourni par le vendeur (en FCFA).
- Traitements : deux calculs.
 - Prix de vente = prix d'achat augmenté de 5 %, c'est-à-dire :

$$\text{prix de vente} = \text{prix d'achat} \times 1,05$$
 - Bénéfice = différence entre le prix de vente et le prix d'achat :

$$\text{bénéfice} = \text{prix de vente} - \text{prix d'achat}$$
- Sorties : le prix de vente et le bénéfice, affichés à l'écran avec des messages clairs.

Étape 2 : choix des variables et de la constante.

Nom	Nature et type	Rôle (justification)
TAUX	Constante Réel = 1,05	Coefficient de la marge : il est fixé à 5 % une fois pour toutes, il ne change jamais pendant l'exécution : c'est donc une constante .
prixAchat	Variable Réel	Prix d'achat saisi par le vendeur : il change à chaque vente. Type Réel car un prix peut avoir des centimes.
prixVente	Variable Réel	Résultat du calcul du prix de vente.
benefice	Variable Réel	Résultat du calcul du bénéfice.

Les noms choisis sont **significatifs** : ils décrivent clairement le contenu, sans espace ni accent, et commencent par une lettre.

Étape 3 : rédaction de l'algorithme complet.

```

Algorithme vente_credit
// Calcule le prix de vente et le benefice d'une vente de credit

Constante
    TAUX : Reel <- 1,05      // marge fixe de 5 %

Variable
    prixAchat : Reel          // prix d'achat saisi (FCFA)
    prixVente : Reel          // prix de vente calcule (FCFA)
    benefice : Reel           // benefice realise (FCFA)

Debut
    // 1. Lecture de l'entree
    Ecrire("Entrez le prix d'achat du credit (FCFA) : ")
    Lire(prixAchat)

    // 2. Traitements
    prixVente <- prixAchat * TAUX
    benefice <- prixVente - prixAchat

    // 3. Affichage des sorties
    Ecrire("Prix de vente : ", prixVente, " FCFA")
    Ecrire("Benefice : ", benefice, " FCFA")
Fin
  
```

Justification de l'ordre des instructions : on doit d'abord Lire le prix d'achat avant tout calcul (sinon `prixAchat` n'a aucune valeur) ; on doit calculer `prixVente` avant `benefice`, car le bénéfice utilise la valeur de `prixVente`. Inverser ces deux affectations provoquerait une erreur : le bénéfice serait calculé avec un prix de vente encore inconnu.

Étape 4 : vérification par tableau de trace (achat de 2 000 FCFA).

Instruction exécutée	prixAchat	prixVente	benefice
Début	indéfini	indéfini	indéfini
Lire(prixAchat)	2 000	indéfini	indéfini
prixVente ← prixAchat × TAUX	2 000	2 100	indéfini
benefice ← prixVente - prixAchat	2 000	2 100	100
Ecrire(...)	2 000	2 100	100

Vérification à la main : $2\,000 \times 1,05 = 2\,100$ FCFA et $2\,100 - 2\,000 = 100$ FCFA. L'algorithme affiche donc :

```

Prix de vente : 2100 FCFA
Benefice : 100 FCFA
  
```

Les résultats de la trace correspondent exactement au calcul manuel : l'algorithme est correct.

Conclusion : pour un achat de crédit à 2 000 FCFA, Armand doit vendre à 2 100 FCFA et gagne 100 FCFA. L'algorithme fonctionne pour n'importe quel prix d'achat saisi : il suffit de relancer l'exécution.

ATTENTION Erreurs fréquentes à éviter

- Oublier de déclarer une variable (ou la constante `TAUX`) avant de l'utiliser.
- Écrire la marge sous la forme 5% dans un calcul : en pseudo-code, on écrit le coefficient 1,05 ou l'expression `prixAchat + prixAchat * 5 / 100`.
- Calculer `benefice` avant `prixVente` : l'ordinateur exécute les instructions **dans l'ordre** ; une variable non encore calculée n'a pas de valeur utilisable.
- Confondre Lire et Écrire : Lire reçoit une donnée de l'utilisateur, Écrire affiche un résultat.

11.6 Bilan de l'activité

Cette activité t'a permis de mobiliser, sur une situation commerciale authentique du quotidien camerounais, l'ensemble des savoirs de la leçon :

- l'**analyse d'un problème** en termes d'entrées, de traitements et de sorties, première étape de toute résolution algorithmique ;
- la **structure de base d'un algorithme** : en-tête, déclarations, corps entre Début et Fin ;
- la distinction entre **variable** et **constante**, et le choix justifié des **types de données** ;
- les trois **instructions de base** : lecture, affectation et écriture, dont l'ordre conditionne la justesse du programme ;
- la **vérification par un tableau de trace**, méthode simple et puissante pour tester un algorithme « à la main » avant de le faire exécuter par une machine.

Tu as également constaté qu'un même raisonnement mathématique (augmenter de 5 %) devient, une fois traduit en algorithme, un outil réutilisable à l'infini : Armand peut désormais traiter toutes ses ventes de la journée sans risque d'erreur de calcul. C'est exactement cette démarche — observer un besoin réel, le modéliser, rédiger un algorithme, le vérifier puis justifier sa démarche — qui est attendue de toi au BEPC.

EXERCICE 6 Pour aller plus loin

Reprends l'algorithme de la vente de crédit et modifie-le pour qu'il demande en plus le nombre de ventes identiques réalisées dans la journée, puis qu'il affiche le bénéfice total de la journée. Vérifie ta solution par un tableau de trace avec 3 ventes d'un crédit acheté

1 000 FCFA chacune.

CORRIGÉ Exercice Pour aller plus loin

Énoncé. Modifier l'algorithme précédent pour prendre en compte le nombre de ventes identiques et afficher le bénéfice total. Tracer pour 3 ventes à 1 000 FCFA l'achat.

Solution.

Analyse :

- Entrées : prixAchat (Réel), nbVentes (Entier).
- Traitements : Calcul du prix de vente, du bénéfice unitaire, puis du bénéfice total ($\text{beneficeTotal} \leftarrow \text{beneficeUnitaire} * \text{nbVentes}$).
- Sorties : prixVente, beneficeUnitaire, beneficeTotal.

Algorithme modifié :

Algorithme vente_credit_journee

// Calcule le benefice total d'une journee pour des ventes identiques

Constante

TAUX : Reel $\leftarrow$ 1,05

Variable

prixAchat : Reel
nbVentes : Entier
prixVente : Reel
beneficeUnitaire : Reel
beneficeTotal : Reel

Debut

Ecrire("Entrez le prix d'achat du credit (FCFA) : ")
Lire(prixAchat)
Ecrire("Entrez le nombre de ventes identiques : ")
Lire(nbVentes)

prixVente $\leftarrow$ prixAchat * TAUX
beneficeUnitaire $\leftarrow$ prixVente - prixAchat
beneficeTotal $\leftarrow$ beneficeUnitaire * nbVentes

Ecrire("Prix de vente unitaire : ", prixVente, " FCFA")
Ecrire("Bénéfice unitaire : ", beneficeUnitaire, " FCFA")
Ecrire("Bénéfice total de la journee : ", beneficeTotal, " FCFA")

Fin

Tableau de trace (prixAchat = 1 000, nbVentes = 3) :

Instruction	prixAchat	nbVentes	prixVente	beneficeUnitaire	beneficeTotal
Début	indéfini	indéfini	indéfini	indéfini	indéfini
Lire(prixAchat)	1 000	indéfini	indéfini	indéfini	indéfini
Lire(nbVentes)	1 000	3	indéfini	indéfini	indéfini
$\text{prixVente} \leftarrow \text{prixAchat} \times \text{TAUX}$	1 000	3	1 050	indéfini	indéfini
$\text{beneficeUnitaire} \leftarrow \text{prixVente} - \text{prixAchat}$	1 000	3	1 050	50	indéfini
$\text{beneficeTotal} \leftarrow \text{beneficeUnitaire} \times \text{nbVentes}$	1 000	3	1 050	50	150
Ecrire(...)	1 000	3	1 050	50	150

Vérification : $1\,000 \times 1,05 = 1\,050$; $1\,050 - 1\,000 = 50$; $50 \times 3 = 150$. L'algorithme affiche un bénéfice total de 150 FCFA. Correct.

12 MÉTHODES ET EXERCICES RÉSOLUS

13 PRÉSENTATION DU LANGAGE DE DÉFINITION DES ALGORITHMES

13.1 Méthode 1 : Identifier et rédiger la structure de base d'un algorithme

MÉTHODE Rédiger la structure de base d'un algorithme

Pour écrire un algorithme correctement structuré, suis toujours les étapes suivantes :

1. Nommer l'algorithme : commence par le mot-clé **Algorithme** suivi d'un nom simple, sans espace ni accent (ex. : **Algorithme Calcul_Moyenne**).
2. Déclarer les constantes et les variables : sous les mots-clés **Constantes** puis **Variables**, liste chaque objet avec son type (**Entier**, **Réel**, **Chaîne**, **Caractère**, **Booléen**). Une constante reçoit sa valeur immédiatement (ex. **TVA = 19,25**).
3. Ouvrir le corps par le mot-clé **Début**.
4. Écrire les instructions dans l'ordre logique impératif : lecture des données (**Lire**), traitements (affectations), affichage des résultats (**Écrire**).
5. Fermer l'algorithme par le mot-clé **Fin**.

Réflexes à retenir : chaque variable utilisée doit avoir été déclarée ; les données doivent être lues avant d'être utilisées ; le résultat doit être affiché avant le **Fin**.

EXERCICE RÉSOLU 7 Structure d'un algorithme de calcul de moyenne

Énoncé. Le professeur d'informatique du collège de Bafoussam veut un algorithme qui lit les deux notes d'un élève (sur 20) et affiche sa moyenne. Rédige cet algorithme en respectant la structure de base.

SOLUTION

Solution détaillée.

Étape 1 : analyse du problème.

- Données d'entrée : deux notes, qui peuvent avoir des décimales $\Rightarrow$ type **Réel**.
- Traitement : $\text{moyenne} = \frac{\text{note1} + \text{note2}}{2}$.

- Sortie : la moyenne, de type Réel.

Étape 2 : rédaction en respectant la structure de base.

```

Algorithme Calcul_Moyenne
Variables
    note1, note2, moyenne : Reel
Debut
    Ecrire("Entrez la premiere note : ")
    Lire(note1)
    Ecrire("Entrez la deuxieme note : ")
    Lire(note2)
    moyenne <- (note1 + note2) / 2
    Ecrire("La moyenne est : ", moyenne)
Fin
  
```

Étape 3 : vérification. Si $\text{note1} = 12$ et $\text{note2} = 15$, alors $\text{moyenne} = \frac{12 + 15}{2} = 13,5$. L'algorithme affiche : « La moyenne est : 13,5 ».

Conclusion. L'algorithme respecte les quatre parties de la structure de base : l'en-tête (Algorithme), la déclaration des variables, le corps délimité par Debut et Fin, avec les instructions dans l'ordre lecture → traitement → écriture.

13.2 Méthode 2 : Choisir le bon type de donnée

MÉTHODE Déclarer variables et constantes avec le bon type

1. Repère la nature de la donnée dans l'énoncé : nombre entier, nombre à virgule, texte, seul caractère, oui/non.
2. Associe le type selon le tableau de correspondance :

Type	Nature de la donnée	Exemples
Entier	Nombre sans virgule	âge, effectif d'une classe, quantité
Réel	Nombre à virgule	prix, moyenne, taille, température
Chaîne	Suite de caractères (texte)	nom, adresse, nom d'une ville
Caractère	Un seul caractère	une lettre, 'O' ou 'N'
Booléen	Vrai ou Faux uniquement	réponse à une question oui/non

3. Distingue variable et constante : une **constante** ne change jamais pendant l'exécution (ex. : la TVA de 19,25 %), une **variable** peut changer de valeur.
4. Déclare les constantes avec leur valeur dès le départ, les variables avec leur type seulement.

EXERCICE RÉSOLU 8 Choix des types pour une boutique

Énoncé. Mme Fotso tient une boutique à Douala. Elle veut un algorithme qui gère les informations suivantes : le nom d'un article, son prix unitaire, la quantité en stock, et le fait que l'article soit ou non en promotion. Pour chaque information, indique le type de donnée approprié en justifiant, puis écris la partie déclaration de l'algorithme.

SOLUTION

Solution détaillée.

Analyse de chaque information :

- Nom de l'article : c'est un texte composé de plusieurs caractères (ex. : « Savon ») ⇒ type **Chaîne**.
- Prix unitaire : c'est un nombre qui peut avoir des décimales (ex. : 250,50 FCFA) ⇒ type **Réel**.
- Quantité en stock : on compte des articles entiers (on ne vend pas un demi-savon en stock) ⇒ type **Entier**.
- En promotion : la réponse est seulement Vrai ou Faux ⇒ type **Booléen**.

Partie déclaration de l'algorithme :

```
Algorithme Gestion_Boutique
Variables
  nomArticle : Chaîne
  prixUnitaire : Réel
  quantiteStock : Entier
  enPromotion : Booléen
Debut
  ...
Fin
```

Conclusion. Le choix du type dépend de la nature de la donnée : Chaîne pour le texte, Réel pour les nombres à virgule, Entier pour les nombres entiers, Booléen pour les valeurs vrai/faux. Un mauvais choix de type (par exemple Entier pour un prix) entraînerait la perte des décimales.

13.3 Méthode 3 : Utiliser les instructions de lecture et d'écriture**MÉTHODE** Communiquer avec l'utilisateur : Lire et Écrire

1. Avant chaque `Lire`, affiche un message avec `Ecrire` pour indiquer à l'utilisateur ce qu'il doit saisir : c'est la règle de convivialité.
2. Syntaxe de lecture : `Lire(nom_variable)` — la valeur tapée au clavier est rangée dans

la variable, qui doit avoir été déclarée.

3. **Syntaxe d'écriture** : `Ecrire("message")` pour un texte fixe, `Ecrire(variable)` pour une valeur, ou les deux combinés : `Ecrire("Le total est : ", total)`.
4. **Ordre obligatoire** : on ne peut afficher ou utiliser une variable qu'après lui avoir donné une valeur (par `Lire` ou par affectation).
5. Vérifie que le type de la valeur saisie correspond au type déclaré de la variable.

EXERCICE RÉSOLU 9 Lecture et écriture : facture d'un cybercafé

Énoncé. Un cybercafé de Yaoundé facture la connexion Internet à 300 F l'heure. Écris un algorithme qui demande le nom du client et le nombre d'heures de connexion, puis affiche le nom du client et le montant à payer.

SOLUTION

Solution détaillée.

Étape 1 : identification des données.

- Entrées : nom du client (Chaîne), nombre d'heures (Entier).
- Constante : le tarif horaire, 300 F, qui ne change pas $\Rightarrow$ constante Entier.
- Sortie : montant à payer (Entier, car $300 \times$ heures donne un entier).

Étape 2 : rédaction.

```
Algorithme Facture_Cybercafe
Constantes
    TARIF = 300
Variables
    nomClient : Chaîne
    heures, montant : Entier
Debut
    Ecrire("Entrez le nom du client : ")
    Lire(nomClient)
    Ecrire("Entrez le nombre d'heures : ")
    Lire(heures)
    montant  $\leftarrow$  TARIF * heures
    Ecrire("Client : ", nomClient)
    Ecrire("Montant à payer : ", montant, " FCFA")
Fin
```

Étape 3 : test. Pour le client « Amina » qui reste 3 heures : $\text{montant} = 300 \times 3 = 900$. L'écran affiche : « Client : Amina » puis « Montant à payer : 900 FCFA ».

Conclusion. Chaque `Lire` est précédé d'un `Ecrire` qui guide l'utilisateur, et le résultat est affiché avec un message clair accompagné de l'unité (FCFA).

13.4 Méthode 4 : Maîtriser l'affectation et l'ordre des instructions

MÉTHODE Utiliser correctement l'affectation

1. **Syntaxe** : `variable <- expression`. La flèche va toujours vers la variable : on calcule d'abord l'expression à droite, puis on range le résultat dans la variable à gauche.
2. **Sens unique** : `a <- b` copie la valeur de `b` dans `a` ; ce n'est pas une égalité mathématique, `b` ne change pas.
3. **Écrasement** : une nouvelle affectation remplace l'ancienne valeur de la variable, qui est définitivement perdue.
4. **Incrémentement** : `compteur <- compteur + 1` signifie « la nouvelle valeur de compteur est l'ancienne valeur plus 1 ».
5. **Trace d'exécution** : pour vérifier un algorithme, construis un tableau donnant la valeur de chaque variable après chaque instruction.

EXERCICE RÉSOLU 10 Trace d'exécution et échange de deux variables

Énoncé. On considère l'algorithme suivant :

```
Algorithme Mystere
Variables
  a, b : Entier
Debut
  a <- 5
  b <- 8
  a <- a + b
  b <- a - b
  a <- a - b
  Ecrire(a, b)
Fin
```

1. Construis la table d'exécution de cet algorithme.
2. Que réalise cet algorithme ?

SOLUTION

Solution détaillée.

1. Table d'exécution. On note la valeur de chaque variable après chaque instruction :

Instruction	Valeur de a	Valeur de b
a ← 5	5	indéfini
b ← 8	5	8
a ← a + b	5 + 8 = 13	8
b ← a - b	13	13 - 8 = 5
a ← a - b	13 - 5 = 8	5

L'algorithme affiche donc : 8 5.

2. Rôle de l'algorithme. Au départ, $a = 5$ et $b = 8$; à la fin, $a = 8$ et $b = 5$. Les valeurs de a et b ont été échangées, sans utiliser de variable intermédiaire.

Conclusion. La table d'exécution est l'outil indispensable pour comprendre et vérifier un algorithme : elle montre pas à pas comment l'affectation modifie le contenu des variables. Ici, elle prouve que la suite des trois affectations réalise bien un échange.

13.5 Méthode 5 : Rédiger un algorithme complet à partir d'une situation concrète

MÉTHODE De la situation de vie à l'algorithme complet

Pour résoudre un problème concret par un algorithme, applique la démarche en 5 étapes :

1. Comprendre le problème : lis l'énoncé et reformule ce qui est demandé.
2. Identifier les données : distingue les entrées (à lire), les constantes (valeurs fixes de l'énoncé) et les sorties (résultats à afficher).
3. Établir les formules de traitement : écris les calculs qui transforment les entrées en sorties.
4. Rédiger l'algorithme en respectant la structure de base : en-tête, déclarations, Debut ... Fin.
5. Tester avec des valeurs simples : fais une trace d'exécution et vérifie que le résultat affiché est correct.

Justification de la démarche : chaque étape doit pouvoir être expliquée — au BEPC, la démarche compte autant que le résultat final.

EXERCICE RÉSOLU 11 Algorithme complet : bulletin d'un élève

Énoncé. Au collège de Buea, la moyenne d'un élève en informatique est calculée à partir de trois notes sur 20 : deux évaluations et un devoir qui compte double. Écris un algorithme complet qui lit le nom de l'élève et ses trois notes, calcule sa moyenne et affiche le

nom de l'élève avec sa moyenne. Justifie ta démarche, puis teste ton algorithme avec les notes 10, 14 et 12.

SOLUTION

Solution détaillée.

Étape 1 : compréhension. On veut la moyenne pondérée de trois notes, le devoir comptant double (coefficient 2).

Étape 2 : identification des données.

- Entrées : nom de l'élève (Chaîne), note1, note2 (évaluations), noteDevoir (Réels).
- Sortie : moyenne (Réel).

Étape 3 : formule de traitement. Le devoir compte double, donc :

$$\text{moyenne} = \frac{\text{note1} + \text{note2} + 2 \times \text{noteDevoir}}{4}$$

Le diviseur est 4 car le total des coefficients est $1 + 1 + 2 = 4$.

Étape 4 : rédaction de l'algorithme.

```
Algorithme Bulletin_Eleve
Variables
  nom : Chaîne
  note1, note2, noteDevoir, moyenne : Reel
Debut
  Ecrire("Entrez le nom de l'eleve : ")
  Lire(nom)
  Ecrire("Entrez la note de la premiere evaluation : ")
  Lire(note1)
  Ecrire("Entrez la note de la deuxieme evaluation : ")
  Lire(note2)
  Ecrire("Entrez la note du devoir : ")
  Lire(noteDevoir)
  moyenne <- (note1 + note2 + 2 * noteDevoir) / 4
  Ecrire("Eleve : ", nom)
  Ecrire("Moyenne : ", moyenne, " / 20")
Fin
```

Étape 5 : test avec les notes 10, 14 et 12.

$$\begin{aligned}\text{moyenne} &= \frac{10 + 14 + 2 \times 12}{4} \\ &= \frac{10 + 14 + 24}{4} \\ &= \frac{48}{4} \\ &= 12\end{aligned}$$

L'algorithme affiche : « Moyenne : 12 / 20 », ce qui est correct.

Conclusion. En suivant la démarche en 5 étapes (comprendre, identifier, modéliser, rédiger, tester), on obtient un algorithme complet, correct et vérifiable, qui répond exactement à la situation posée.

13.6 Méthode 6 : Vérifier et corriger un algorithme

MÉTHODE Relire et corriger un algorithme

Avant de rendre ta copie (ou d'exécuter ton algorithme), effectue ces contrôles systématiques :

1. Structure : l'algorithme a-t-il un nom ? Les mots-clés `Debut` et `Fin` sont-ils présents ?
2. Déclarations : chaque variable utilisée est-elle déclarée, avec un type cohérent ?
3. Initialisation : chaque variable a-t-elle reçu une valeur (par `Lire` ou affectation) avant d'être utilisée dans un calcul ou un affichage ?
4. Ordre des instructions : les lectures précèdent-elles les traitements, et les traitements précèdent-ils les affichages ?
5. Test : refais une trace d'exécution avec des valeurs simples et vérifie le résultat à la main.

EXERCICE RÉSOLU 12 Corriger un algorithme mal rédigé

Énoncé. Un élève a écrit l'algorithme suivant pour calculer le périmètre d'un rectangle :

```
Algorithme Perimetre
Debut
    perimetre <- 2 * (longueur + largeur)
    Ecrire("Le perimetre est : ", perimetre)
    Lire(longueur, largeur)
Fin
```

Releve toutes les erreurs commises, puis réécris l'algorithme corrigé.

SOLUTION

Solution détaillée.

Recherche des erreurs :

1. Aucune variable n'est déclarée : `longueur`, `largeur` et `perimetre` sont utilisées sans avoir été déclarées avec leur type.
2. Ordre des instructions incorrect : le calcul `perimetre <- 2 * (longueur + largeur)` est effectué avant la lecture des valeurs. Au moment du calcul, `longueur` et `largeur` n'ont aucune valeur : le calcul est impossible.
3. Absence de messages de saisie : l'utilisateur ne sait pas ce qu'il doit entrer.

Algorithme corrigé :

```
Algorithme Perimetre_Rectangle
Variables
```

```
longueur, largeur, perimetre : Reel
Debut
  Ecrire("Entrez la longueur : ")
  Lire(longueur)
  Ecrire("Entrez la largeur : ")
  Lire(largeur)
  perimetre <- 2 * (longueur + largeur)
  Ecrire("Le perimetre est : ", perimetre)
Fin
```

Vérification. Pour un terrain de longueur 20 m et de largeur 12 m : périmètre = $2 \times (20 + 12) = 2 \times 32 = 64$ m. L'algorithme affiche « Le perimetre est : 64 », ce qui est exact.

Conclusion. La règle d'or est : déclarer toutes les variables, lire les données avant de les utiliser, calculer ensuite, et afficher en dernier.

ATTENTION Les 5 erreurs qui coûtent des points au BEPC

1. Oublier de déclarer les variables (ou déclarer un mauvais type) : toute variable utilisée doit figurer dans la partie `Variables` avec un type adapté. Mettre un prix en Entier fait perdre les décimales.
2. Utiliser une variable avant de lui avoir donné une valeur : calculer avec `longueur` avant de l'avoir lue est une erreur grave. Respecte l'ordre : lecture → traitement → écriture.
3. Inverser le sens de l'affectation : on écrit `moyenne <- (note1 + note2) / 2` et jamais `(note1 + note2) / 2 <- moyenne`. La variable recevant le résultat est toujours à gauche de la flèche.
4. Oublier les mots-clés de structure : un algorithme sans `Algorithme (nom)`, `Debut` ou `Fin` est incomplet et sanctionné, même si les calculs sont justes.
5. Afficher un résultat brut sans message : écrire seulement `Ecrire(moyenne)` rend l'affichage incompréhensible. Accompagne toujours le résultat d'un texte clair et de l'unité : `Ecrire("La moyenne est : ", moyenne, " / 20")`.

14 EXERCICES

15 RAPPELS ET NOTION DE LANGAGE DE DÉFINITION DES ALGORITHMES

LESAVAIS-TU ? Origine du mot « algorithme »

Le terme **algorithme** vient du nom du mathématicien perse Al-Khwârizmî (vers 780–850), qui a écrit un traité sur le calcul indien et l'algèbre. Son nom, latinisé en Algoritmi, a donné le mot « algorithme » : une méthode de calcul systématique, étape par étape, qui aboutit à un résultat garanti.

DÉFINITION Algorithme

Un algorithme est une suite finie et ordonnée d'instructions précises et non ambiguës, permettant de résoudre un problème ou d'obtenir un résultat à partir de données d'entrée.

À RETENIR Caractéristiques d'un algorithme

Pour être valide, un algorithme doit respecter cinq propriétés fondamentales :

1. Finitude : il se termine après un nombre fini d'étapes.
2. Précision (déterminisme) : chaque instruction est claire et ne laisse place à aucune interprétation.
3. Entrées : zéro ou plusieurs données initiales (fournies par l'utilisateur ou lues dans un fichier).
4. Sorties : une ou plusieurs données résultantes (affichées, stockées ou transmises).
5. Efficacité : chaque instruction doit être réalisable en un temps fini avec les moyens disponibles (pas d'opérations impossibles comme « diviser par zéro » ou « deviner un nombre »).

PROPRIÉTÉ Langage de définition d'algorithmes (Pseudo-code)

Au lieu d'écrire directement dans un langage de programmation (C, Python, JavaScript...), on utilise un langage de définition d'algorithmes (souvent appelé pseudo-code). Ce langage :

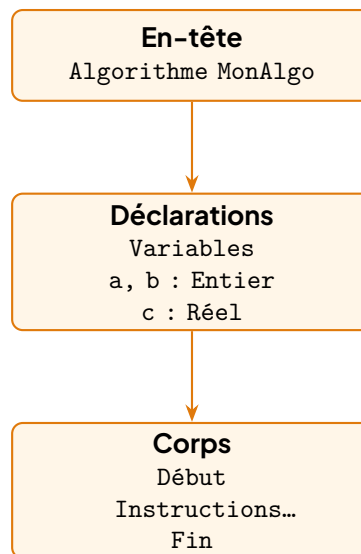
- utilise des mots-clés en français (ou en anglais selon les conventions) : `Algorithme`, `Variable`, `Début`, `Fin`, `Si`, `TantQue`, `Lire`, `Écrire`...
- est indépendant de la machine et du compilateur ;
- permet de se concentrer sur la logique et la structure du programme sans se soucier de la syntaxe stricte d'un langage réel (points-virgules, accolades, includes...).

C'est le langage que nous utiliserons tout au long de l'année pour concevoir nos programmes avant de les coder en C ou en HTML/JavaScript.

16 STRUCTURE DE BASE D'UN ALGORITHME**PROPRIÉTÉ** Les trois parties obligatoires

Tout algorithme écrit en pseudo-code se compose de trois grandes parties, dans cet ordre strict :

1. L'en-tête : commence par le mot-clé `Algorithme` suivi du nom de l'algorithme (sans espace, commençant par une majuscule, ex. : `CalculMoyenne`).
2. La partie déclarative : introduite par le mot-clé `Variables` (ou `Variable`), elle liste toutes les variables et constantes utilisées avec leur type (`Entier`, `Réel`, `Chaîne`, `Booléen`). Optionnel si aucune variable n'est utilisée.
3. Le corps de l'algorithme : délimité par `Début` et `Fin`, il contient la suite des instructions exécutées séquentiellement.



Structure générale d'un algorithme en pseudo-code

EXEMPLE Squelette minimal

Voici la structure la plus simple possible (aucun traitement, aucune variable) :

```
Algorithme Vide  
Début  
    (* Aucune instruction *)  
Fin
```

ATTENTION Erreurs fréquentes sur la structure

- Oublier Début ou Fin.
- Écrire des instructions en dehors du bloc Début ... Fin (par exemple après Fin).
- Mettre les déclarations après Début : les déclarations précèdent toujours Début.
- Utiliser des accents ou des espaces dans le nom de l'algorithme (ex. Algorithme Calcul Moyenne → incorrect ; écrire CalculMoyenne).

17 VARIABLES, CONSTANTES ET TYPES DE DONNÉES

DÉFINITION Variable

Une variable est une zone de mémoire identifiée par un nom (identificateur), qui contient une valeur unique susceptible de changer au cours de l'exécution de l'algorithme.

DÉFINITION Constante

Une constante est une zone de mémoire identifiée par un nom, dont la valeur est fixée une seule fois à la déclaration et ne peut jamais être modifiée pendant l'exécution.

PROPRIÉTÉ Types de données de base (niveau 3ème)

En algorithmique au collège, on utilise quatre types fondamentaux :

Type	Description / Valeurs admises	Exemple de littéral
Entier	Nombres entiers relatifs (... , -2, -1, 0, 1, 2, ...)	42, -7, 0
Réel	Nombres à virgule (notation décimale avec point ou virgule selon convention, ici virgule française)	3,14, -0,5, 12,0
Chaîne	Suite de caractères (lettres, chiffres, symboles) entre guillemets	"Bonjour", "3ème C"
Booléen	Valeur logique : Vrai ou Faux	Vrai, Faux

PROPRIÉTÉ Syntaxe de déclaration

```
Variables
  <nom1>, <nom2> : <Type> ;
  <nom3> : <Type> ;
Constantes
  <NOM_CONST> = <valeur> : <Type> ;
```

- Les noms de variables commencent par une minuscule (ex. note, prixUnitaire).
- Les noms de constantes s'écrivent en MAJUSCULES (ex. TAUX_TVA, ANNEE_COURANTE).
- On sépare les variables de même type par des virgules ; on termine chaque ligne par un point-virgule.

EXEMPLE Déclarations variées

Variables

```
age, nbEleves : Entier ;  
moyenne, prix : Réel ;  
nomEleve, ville : Chaîne ;  
estAdmis : Booléen ;
```

Constantes

```
ANNEE_COURANTE = 2026 : Entier ;  
PI = 3,14159 : Réel ;  
NOM_ETAB = "Lycee Bilingue" : Chaîne ;
```

ATTENTION Règles de nommage (identificateurs)

- Commencer par une lettre (pas de chiffre au début).
- Pas d'espaces, pas d'accents, pas de caractères spéciaux (% , \$, # , @ ...). Le trait de soulignement _ est autorisé.
- Respecter la casse : `maVariable` ≠ `mavariabLe`.
- Choisir des noms significatifs : `noteMaths` est mieux que `n` ou `x`.

18 LES INSTRUCTIONS DE BASE : LECTURE, ÉCRITURE, AFFECTATION

PROPRIÉTÉ Instruction d'affectation : ←

L'affectation stocke une valeur dans une variable.

`<Variable> ← <Expression>`

- La flèche ← (saisie ← au clavier) signifie « reçoit la valeur de ».
- L'<Expression> peut être une valeur littérale, une variable, ou un calcul ($a + b * 2$, `Racine(16)...`).
- Sens unique : la variable cible est à gauche, la valeur source à droite.

PROPRIÉTÉ Instruction de lecture : Lire

Permet de saisir une valeur au clavier pendant l'exécution et de la stocker dans une variable.

Lire(<Variable>)

- La variable doit être déclarée et de type compatible avec la saisie attendue.
- L'exécution suspend l'algorithme jusqu'à la validation (touche Entrée).

PROPRIÉTÉ Instruction d'écriture : Écrire

Affiche une information à l'écran (sortie standard).

Écrire(<Expression>) ou Écrire(<Message>, <Variable>)

- Les chaînes de caractères (messages) sont entre guillemets doubles : "Total : ".
- On peut enchaîner plusieurs arguments séparés par des virgules (selon les dialectes) ou faire plusieurs Écrire successifs. Nous utiliserons plusieurs Écrire pour la clarté.
- Écrire n'affiche pas de retour à la ligne automatique à la fin (sauf convention locale); on gère l'affichage ligne par ligne.

MÉTHODE Rédiger un algorithme simple (3 étapes)

1. Analyser le problème : identifier les entrées (données connues ou demandées), les sorties (résultats attendus) et les traitements (formules, calculs).
2. Concevoir : choisir les variables (noms, types), les constantes, et ordonner les instructions logiquement (Lire → Calculer → Écrire).
3. Coder en pseudo-code : respecter la structure Algorithme / Variables / Début / Fin et la syntaxe des trois instructions de base.

EXEMPLE Algorithme complet : Calcul du prix TTC

Problème : Saisir un prix HT, appliquer un taux de TVA constant (19,25%), afficher le prix TTC.

```
Algorithme PrixTTC
Variables
    prixHT, prixTTC : Réel ;
```

```

Constantes
    TAUX_TVA = 0,1925 : Réel ;
Début
    Écrire("Entrez le prix HT (FCFA) : ")
    Lire(prixHT)
    prixTTC <- prixHT * (1 + TAUX_TVA)
    Écrire("Le prix TTC est : ")
    Écrire(prixTTC)
    Écrire(" FCFA")
Fin

```

19 RÉDACTION COMPLÈTE D'ALGORITHMES SUR DES SITUATIONS CONCRÈTES

EXERCICE RÉSOLU 13 Calcul de la moyenne de trois notes

Énoncé : Écrire un algorithme `Moyenne3Notes` qui lit trois notes sur 20, calcule leur moyenne et l'affiche avec un message.

Analyse :

- Entrées : trois notes (réelles, ex. 12,5 ; 14 ; 9,75).
- Sortie : la moyenne (réelle).
- Traitement : somme des notes divisée par 3.

Algorithme :

```

Algorithme Moyenne3Notes
Variables
    note1, note2, note3, somme, moyenne : Réel ;
Début
    Écrire("Entrez la note 1 : ")
    Lire(note1)
    Écrire("Entrez la note 2 : ")
    Lire(note2)
    Écrire("Entrez la note 3 : ")
    Lire(note3)
    somme <- note1 + note2 + note3
    moyenne <- somme / 3
    Écrire("La moyenne des trois notes est : ")
    Écrire(moyenne)
    Écrire(" / 20")
Fin

```

Test (trace) : Si notes = 12, 14, 10 → somme = 36 → moyenne = 12. Affichage : La moyenne

des trois notes est : 12 / 20.

SOLUTION

Remarques pédagogiques :

- Le type `Réel` est indispensable car les notes peuvent avoir des demi-points.
- La division par 3 (entier) est une division réelle car `somme` est réel.
- L'affichage est découpé pour être lisible.

19.1 Je vérifie mes connaissances

EXERCICE 7 QCM : à chaque question, une seule bonne réponse

Pour chaque question, recopie la lettre de la bonne réponse.

1. Un algorithme commence toujours par le mot-clé :
 - a) Fin
 - b) Début
 - c) Variable
 - d) Écrire
2. L'instruction qui permet à l'utilisateur d'entrer une valeur est :
 - a) Écrire
 - b) Afficher
 - c) Lire
 - d) Calculer
3. Le symbole `←` (saisi `<-`) signifie :
 - a) une comparaison
 - b) une affectation
 - c) une lecture
 - d) une fin d'algorithme
4. La valeur d'une constante au cours de l'exécution d'un algorithme :
 - a) change à chaque étape
 - b) ne change jamais
 - c) change seulement à la fin

d) est toujours nulle

EXERCICE 8 Vrai ou faux ? Justifie ta réponse

Réponds par vrai ou faux, puis justifie en une phrase.

1. Une variable doit être déclarée avant d'être utilisée dans un algorithme.
2. L'instruction `Écrire("Bonjour")` permet de saisir le prénom de l'utilisateur.
3. Une variable de type `Entier` peut contenir la valeur 12,75.
4. Dans l'instruction `A ← B`, c'est la variable A qui reçoit la valeur de B.

EXERCICE 9 Définitions

Définis en une ou deux phrases chacun des termes suivants, puis donne un exemple pour chacun :

1. Algorithme ;
2. Variable ;
3. Constante ;
4. Affectation.

EXERCICE 10 Rôles des instructions de base

Recopie et complète le tableau ci-dessous en associant à chaque instruction son rôle, puis donne pour chacune un exemple d'utilisation dans un algorithme de la vie courante (cantine scolaire, marché, transport en moto-taxi...).

Instruction	Rôle	Exemple d'utilisation
Lire		
Écrire		
← (affectation)		

19.2 Je m'entraîne

EXERCICE 11 Au marché de Mokolo

Écris en pseudo-code un algorithme nommé `TotalAchat` qui :

1. lit le prix unitaire (en FCFA) et la quantité d'un article acheté au marché ;
2. calcule le montant total à payer ;
3. affiche le montant total avec un message clair.

Tu déclareras toutes les variables utilisées en précisant leur type.

EXERCICE 12 Année de naissance

Écris en pseudo-code un algorithme nommé `Naissance` qui lit l'âge d'un élève de 3ème et affiche son année de naissance, sachant que l'année courante est 2026. Tu utiliseras une constante pour l'année courante et tu justifieras ce choix.

EXERCICE 13 Périmètre et aire d'un carré

Le terrain de sport de ton collège a la forme d'un carré.

1. Écris en pseudo-code un algorithme qui lit la longueur du côté du terrain (en mètres), puis calcule et affiche son périmètre et son aire.
2. Justifie le choix du type (`Entier` ou `Réel`) de chaque variable utilisée.

EXERCICE 14 Tableau de trace

On considère l'algorithme suivant :

```
Algorithme Mystere
Variable a, b, c : Entier
Début
    a <- 5
    b <- 3
    c <- a + b
    a <- c - b
    Écrire(a)
    Écrire(c)
Fin
```

1. Recopie et complète le tableau de trace ci-dessous en donnant la valeur de chaque variable après l'exécution de chaque ligne.

Instruction exécutée	a	b	c
a ← 5			
b ← 3			
c ← a + b			
a ← c - b			

2. Déduis-en les valeurs affichées par l'algorithme à la fin de son exécution.

19.3 Je me prépare au BEPC

EXERCICE 15 Type BEPC : la chasse aux erreurs

Marius, élève en classe de 3ème au collège bilingue de Bafoussam, a rédigé l'algorithme ci-dessous pour calculer la moyenne de deux notes d'un camarade. Malheureusement, son algorithme contient trois erreurs : une variable non déclarée, un mauvais type de variable et une affectation inversée (sens de la flèche).

```

Algorithme MoyenneNotes
Variable note1, note2 : Entier
Variable moyenne : Entier
Début
    Écrire("Entrez la première note : ")
    Lire(note1)
    Écrire("Entrez la deuxième note : ")
    Lire(note2)
    note1 + note2 ← somme
    moyenne ← somme / 2
    Écrire("La moyenne est : ")
    Écrire(moyenne)
Fin
  
```

1. Recopie la structure de base d'un algorithme (les trois grandes parties) en les nommant.
2. Identifie les trois erreurs commises par Marius en précisant la ligne concernée et la nature de chaque erreur.
3. Réécris l'algorithme complet corrigé, en choisissant des types de variables adaptés (une moyenne peut être un nombre à virgule, par exemple 12,5).
4. Donne la valeur affichée par l'algorithme corrigé si l'utilisateur entre les notes 13 et 17.

EXERCICE 16 Type BEPC : la boutique de Mme Etoundi

Mme Etoundi tient une boutique d'articles scolaires à Yaoundé. Pour la rentrée, elle souhaite disposer d'un programme simple qui l'aide à calculer la facture d'un client. Une règle de sa boutique est fixe : le prix d'un paquet de 10 stylos est de 1000 FCFA, et ce prix ne change jamais au cours de l'année.

1. Explique pourquoi le prix du paquet de stylos doit être représenté par une constante et non par une variable, puis donne la ligne de déclaration correspondante en pseudo-code.
2. Écris en pseudo-code un algorithme nommé *Facture* qui :
 - lit le nombre de paquets de stylos achetés par le client ;
 - lit le montant des autres articles achetés (cahiers, règles...) ;
 - calcule le montant total de la facture ;
 - affiche le montant total à payer avec un message poli du type : « Merci, vous devez payer ... FCFA ».
3. Mme Etoundi vend 4 paquets de stylos et pour 2500 FCFA d'autres articles à un client. Fais le tableau de trace de ton algorithme et donne le message exact affiché à l'écran.
4. Rédige une conclusion de deux phrases expliquant l'intérêt d'un tel algorithme pour la gestion quotidienne de la boutique.

EXERCICE 17 Type BEPC : le transport scolaire

Le collège de Nkongsamba organise le transport de ses élèves par cars. Chaque car peut transporter au maximum 50 élèves. Le proviseur demande à la classe de 3ème de concevoir un algorithme qui, à partir du nombre d'élèves inscrits à une sortie pédagogique, affiche le nombre de cars complets et le nombre d'élèves restant dans le dernier car. On rappelle que :

- **div** désigne la division entière : $137 \text{ div } 50 = 2$;
 - **mod** désigne le reste de la division entière : $137 \text{ mod } 50 = 37$.
1. Donne la définition d'un algorithme et cite les trois instructions de base étudiées en classe.
 2. Identifie les données d'entrée et les données de sortie du problème posé, puis propose un nom et un type pour chaque variable (le nombre de cars et le nombre d'élèves restants sont des nombres entiers).
 3. Écris en pseudo-code l'algorithme *Transport* qui lit le nombre d'élèves inscrits, calcule le nombre de cars complets et le nombre d'élèves restants, puis affiche les deux

résultats avec des messages explicites.

4. Vérifie ton algorithme en faisant son tableau de trace pour 137 élèves inscrits, puis pour 100 élèves inscrits. Que remarques-tu dans le second cas ?
5. Justifie en une phrase le choix du type `Entier` pour toutes les variables de cet algorithme.

20 CORRIGÉS DES EXERCICES

CORRIGÉ Exercice 1 — QCM : à chaque question, une seule bonne réponse

1. Réponse b) : `Début`. Le corps de l'algorithme, qui contient les instructions, commence toujours par le mot-clé `Début` et se termine par `Fin`. (Algorithme ouvre l'en-tête, mais le corps commence par `Début`.)
2. Réponse c) : `Lire`. C'est l'instruction de lecture (saisie au clavier) : `Lire(variable)`.
3. Réponse b) : une affectation. `A <- B` signifie « A reçoit la valeur de B ».
4. Réponse b) : ne change jamais. Par définition, la valeur d'une constante est fixée une seule fois à la déclaration et ne peut pas être modifiée pendant l'exécution.

CORRIGÉ Exercice 2 — Vrai ou faux ? Justifie ta réponse

1. Vrai. Toute variable doit être déclarée (avec son type) dans la partie déclarative avant le mot-clé `Début` ; sinon l'algorithme ne sait pas quel type de valeur elle contient.
2. Faux. `Écrire("Bonjour")` est une instruction d'affichage : elle affiche un message à l'écran. Pour saisir le prénom de l'utilisateur, il faut utiliser `Lire(prenom)`.
3. Faux. Une variable de type `Entier` ne peut contenir que des nombres entiers (... , -1, 0, 1, 2, ...). La valeur 12,75 possède une partie décimale : il faut un type `Réel`.
4. Vrai. Dans une affectation, la variable cible est toujours à gauche de la flèche et la valeur source à droite : `A <- B` place la valeur de B dans A.

CORRIGÉ Exercice 3 — Définitions

1. Algorithme : suite finie et ordonnée d'instructions précises et non ambiguës permettant de résoudre un problème. Exemple : l'algorithme `Moyenne3Notes` qui calcule la moyenne de trois notes.
2. Variable : zone de mémoire identifiée par un nom, contenant une valeur qui peut changer au cours de l'exécution. Exemple : `note1 : Réel`.
3. Constante : zone de mémoire identifiée par un nom, dont la valeur est fixée à la déclaration et ne change jamais pendant l'exécution. Exemple : `ANNEE_COURANTE = 2026 : Entier`.

4. **Affectation** : instruction qui stocke une valeur dans une variable, notée `Variable <- Expression`. Exemple : `total <- prix * quantite`.

CORRIGÉ Exercice 4 — Rôles des instructions de base

Instruction	Rôle	Exemple d'utilisation
Lire	Saisir une valeur au clavier et la stocker dans une variable	<code>Lire(prix)</code> : le commerçant entre le prix d'un article au marché
Écrire	Afficher un message ou un résultat à l'écran	<code>Écrire("Total à payer : ")</code> à la cantine scolaire
<code><-</code> (affectation)	Stocker une valeur ou le résultat d'un calcul dans une variable	<code>total <- prix * quantite</code> pour calculer le montant d'un achat

CORRIGÉ Exercice 5 — Au marché de Mokolo

Analyse : entrées : prix unitaire et quantité ; sortie : montant total ; traitement : `total <- prixUnitaire * quantite`.

Algorithme TotalAchat

Variables

`prixUnitaire` : Reel ;

`quantite` : Entier ;

`total` : Reel ;

Début

`Écrire("Entrez le prix unitaire (FCFA) : ")`

`Lire(prixUnitaire)`

`Écrire("Entrez la quantité : ")`

`Lire(quantite)`

`total <- prixUnitaire * quantite`

`Écrire("Le montant total à payer est : ")`

`Écrire(total)`

`Écrire(" FCFA")`

Fin

Justification des types : `prixUnitaire` et `total` sont de type `Réel` car un prix peut avoir des centimes ; `quantite` est un `Entier` car on compte des articles entiers.

CORRIGÉ Exercice 6 — Année de naissance

```
Algorithme Naissance
Variables
    age, anneeNaissance : Entier ;
Constantes
    ANNEE_COURANTE = 2026 : Entier ;
Début
    Écrire("Entrez votre âge : ")
    Lire(age)
    anneeNaissance <- ANNEE_COURANTE - age
    Écrire("Vous êtes né(e) en : ")
    Écrire(anneeNaissance)
Fin
```

Justification : l'année courante est déclarée en constante car elle est fixe pendant toute l'exécution (2026) et ne doit pas être modifiée par l'algorithme ; l'écrire en constante évite toute modification accidentelle et rend l'algorithme plus lisible.

CORRIGÉ Exercice 7 — Périmètre et aire d'un carré

1. Algorithme :

```
Algorithme TerrainCarre
Variables
    cote, perimetre, aire : Reel ;
Début
    Écrire("Entrez la longueur du côté (m) : ")
    Lire(cote)
    perimetre <- 4 * cote
    aire <- cote * cote
    Écrire("Le périmètre est : ")
    Écrire(perimetre)
    Écrire(" m")
    Écrire("L'aire est : ")
    Écrire(aire)
    Écrire(" m²")
Fin
```

2. Justification des types : le type `Reel` est choisi pour les trois variables car la longueur du côté peut être un nombre à virgule (par exemple 25,5 m), et le périmètre et l'aire, calculés à partir d'elle, peuvent donc aussi être décimaux. Si l'on était sûr que le côté est toujours entier, on pourrait utiliser `Entier`.

CORRIGÉ Exercice 8 — Tableau de trace

1. Tableau de trace complété :

Instruction exécutée	a	b	c
a ← 5	5	—	—
b ← 3	5	3	—
c ← a + b	5	3	8
a ← c - b	5	3	8

Détail du dernier calcul : $a \leftarrow c - b$ donne $a = 8 - 3 = 5$. La nouvelle valeur de a est donc 5 (elle remplace l'ancienne, qui était déjà 5).

2. Valeurs affichées : `Écrire(a)` affiche 5, puis `Écrire(c)` affiche 8.

CORRIGÉ Exercice 9 — Type BEPC : la chasse aux erreurs

1. Les trois grandes parties d'un algorithme : l'en-tête (Algorithme NomAlgo), la partie déclarative (Variables / Constantes) et le corps (entre Début et Fin).

2. Les trois erreurs :

- Ligne `note1 + note2 ← somme` : affectation inversée — la variable cible doit être à gauche ; on doit écrire `somme ← note1 + note2`.
- Ligne `moyenne ← somme / 2` : la variable `somme` est non déclarée dans la partie déclarative.
- Ligne `Variable moyenne : Entier` : mauvais type — une moyenne peut être un nombre à virgule (ex. 12,5), il faut le type `Réel`.

3. Algorithme corrigé :

```

Algorithme MoyenneNotes
Variables
    note1, note2 : Réel ;
    somme, moyenne : Réel ;
Début
    Écrire("Entrez la première note : ")
    Lire(note1)
    Écrire("Entrez la deuxième note : ")
    Lire(note2)
    somme ← note1 + note2
    moyenne ← somme / 2
    Écrire("La moyenne est : ")
    Écrire(moyenne)
Fin
  
```

4. Avec les notes 13 et 17 : $\text{somme} = 13 + 17 = 30$; $\text{moyenne} = 30 / 2 = 15$. L'algorithme affiche : La moyenne est : 15.

Barème indicatif (10 pts) : structure (2 pts) ; trois erreurs identifiées avec nature (3 pts) ; algorithme corrigé complet (3 pts) ; valeur affichée justifiée (2 pts). Points de vigilance : citer explicitement la règle « la cible de l'affectation est à gauche de la flèche » ; déclarer somme ; justifier le type `Réel` par l'existence possible de demi-points.

CORRIGÉ Exercice 10 — Type BEPC : la boutique de Mme Etoundi

1. Le prix du paquet de stylos (1000 FCFA) ne change jamais au cours de l'année : c'est une valeur fixe, donc une constante. La déclarer en constante garantit qu'aucune instruction ne pourra la modifier par erreur.

```
Constantes
    PRIX_PAQUET = 1000 : Entier ;
```

2. Algorithme :

```
Algorithme Facture
Variables
    nbPaquets : Entier ;
    autresArticles, total : Reel ;
Constantes
    PRIX_PAQUET = 1000 : Entier ;
Début
    Écrire("Nombre de paquets de stylos : ")
    Lire(nbPaquets)
    Écrire("Montant des autres articles (FCFA) : ")
    Lire(autresArticles)
    total <- nbPaquets * PRIX_PAQUET + autresArticles
    Écrire("Merci, vous devez payer ")
    Écrire(total)
    Écrire(" FCFA")
Fin
```

3. Tableau de trace pour 4 paquets et 2500 FCFA d'autres articles :

Instruction	nbPaquets	autresArticles	total
Lire(nbPaquets)	4	—	—
Lire(autresArticles)	4	2500	—
total <- ...	4	2500	6500

Calcul : $4 \times 1000 + 2500 = 4000 + 2500 = 6500$. Message affiché : « Merci, vous devez payer 6500 FCFA ».

4. Conclusion : cet algorithme permet à Mme Etoundi de calculer rapidement et sans erreur la facture de chaque client. Il illustre comment l'informatique automatise les tâches répétitives de la vie quotidienne et fiabilise la gestion d'une boutique.

Barème indicatif (10 pts) : justification de la constante + déclaration (2 pts) ; algorithme complet et correct (4 pts) ; trace et message exact (2 pts) ; conclusion rédigée (2 pts). Points de vigilance : la constante doit être déclarée avant **Début** ; le message affiché doit respecter exactement la formulation demandée ; le calcul doit être détaillé.

CORRIGÉ Exercice 11 — Type BEPC : le transport scolaire

1. Un algorithme est une suite finie et ordonnée d'instructions précises et non ambiguës permettant de résoudre un problème. Les trois instructions de base sont : Lire (saisie), Écrire (affichage) et l'affectation $\leftarrow$ (calcul et stockage).
2. Entrée : le nombre d'élèves inscrits : `nbEleves` : Entier. Sorties : le nombre de cars complets : `nbCars` : Entier ; le nombre d'élèves restants : `restants` : Entier.
3. Algorithme :

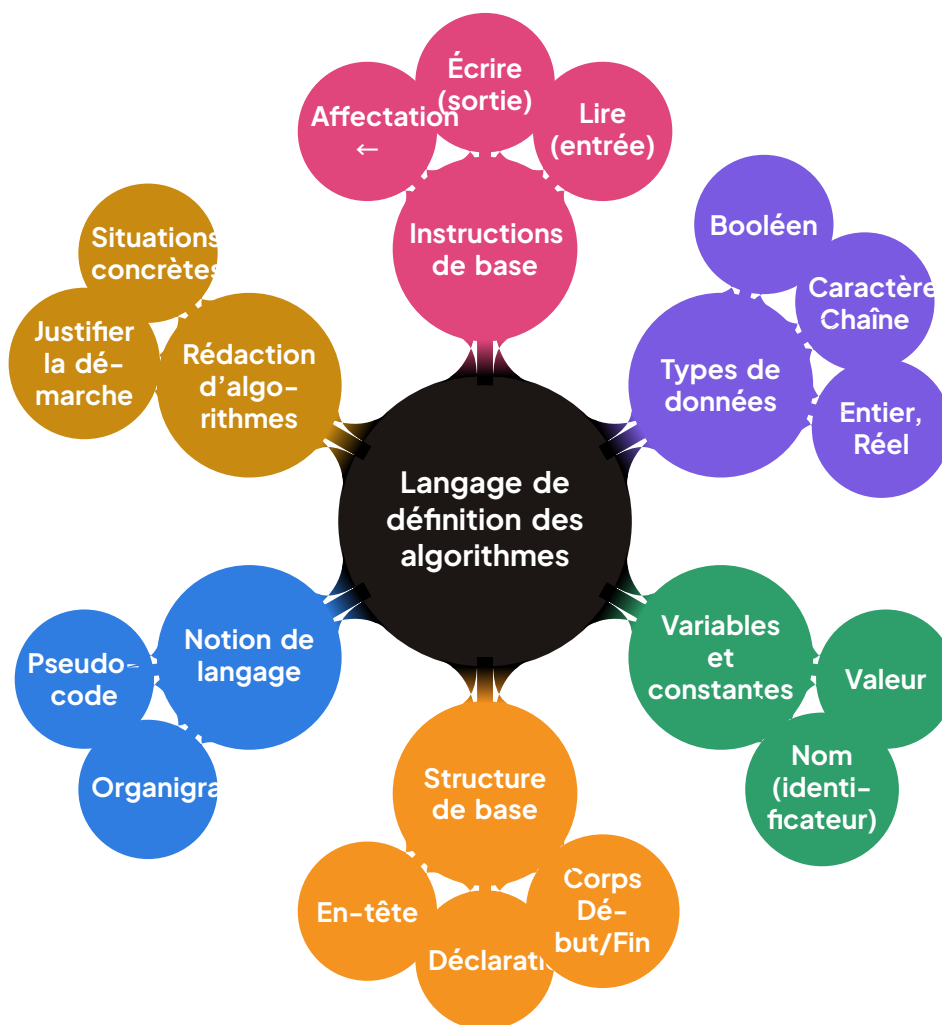
```

Algorithme Transport
Variables
    nbEleves, nbCars, restants : Entier ;
Constantes
    CAPACITE = 50 : Entier ;
Début
    Écrire("Entrez le nombre d'élèves inscrits : ")
    Lire(nbEleves)
    nbCars  $\leftarrow$  nbEleves div CAPACITE
    restants  $\leftarrow$  nbEleves mod CAPACITE
    Écrire("Nombre de cars complets : ")
    Écrire(nbCars)
    Écrire("Élèves dans le dernier car : ")
    Écrire(restants)
Fin
  
```

4. Trace pour 137 élèves : $137 \text{ div } 50 = 2$ (car $2 \times 50 = 100 \leq 137$) et $137 \text{ mod } 50 = 137 - 100 = 37$. Affichage : 2 cars complets, 37 élèves restants. Vérification : $2 \times 50 + 37 = 137$. ✓
Trace pour 100 élèves : $100 \text{ div } 50 = 2$ et $100 \text{ mod } 50 = 0$. Affichage : 2 cars complets, 0 élève restant. Remarque : dans ce cas, il n'y a pas de dernier car partiellement rempli : tous les cars sont complets, le reste est nul.
5. Toutes les variables sont de type `Entier` car on compte des élèves et des cars, qui sont des quantités entières (on ne peut pas avoir 2,5 cars ou 37,5 élèves), et les opérateurs `div` et `mod` ne s'appliquent qu'à des entiers.

Barème indicatif (10 pts) : définition + instructions de base (2 pts) ; entrées/sorties avec noms et types (2 pts) ; algorithme correct avec `div` et `mod` (3 pts) ; deux traces vérifiées + remarque (2 pts) ; justification du type (1 pt). Points de vigilance : bien distinguer `div` (quotient) et `mod` (reste) ; vérifier chaque trace par la relation $\text{nbEleves} = \text{nbCars} \times 50 + \text{restants}$; traiter le cas particulier où le reste vaut 0.

21 FICHE BILAN



Carte mentale de la leçon : le langage de définition des algorithmes.

FICHE BILAN L'essentiel à connaître pour l'examen

Définitions clés

- Un **algorithme** est une suite finie et ordonnée d'instructions permettant de résoudre un problème.
- Le **langage de définition des algorithmes** (ou pseudo-code) est un langage simple, en français, qui permet d'écrire un algorithme indépendamment de tout langage de programmation.
- Une **variable** est une zone mémoire nommée dont la valeur peut changer ; une

constante garde toujours la même valeur.

- L'**affectation** (symbole $\leftarrow$) donne une valeur à une variable : $x \leftarrow 5$.

Structure de base d'un algorithme (à connaître par cœur)

```

Algorithme nom_de_l_algorithme
Variable x : Entier      (* déclarations *)
Constante TVA : Reel  $\leftarrow$  19,25
Debut
    Ecrire("Entrez une valeur : ")
    Lire(x)              (* entrée *)
     $x \leftarrow x + 1$       (* traitement *)
    Ecrire("Résultat : ", x) (* sortie *)
Fin
  
```

Types de données et instructions

Type	Exemples de valeurs	Usage
Entier	-3, 0, 25	âges, quantités, notes entières
Réel	3,14; 19,25	prix, moyennes, mesures
Caractère	'A', 'z'	une seule lettre ou un symbole
Chaîne	"Yaoundé"	noms, mots, phrases
Booléen	Vrai, Faux	réponses oui/non, tests

Méthode express pour rédiger un algorithme

1. Comprendre le problème : quelles sont les **données** (entrées) et le **résultat** attendu (sortie)?
2. Choisir et déclarer les variables avec le bon type.
3. Écrire les instructions dans l'ordre : Ecrire/Lire (entrées), calculs par affectation (traitement), Ecrire (sortie).
4. Vérifier en exécutant l'algorithme « à la main » avec des valeurs d'exemple.

À RETENIR

Checklist avant le BEPC

- ☐ Je sais définir un algorithme et dire à quoi sert le pseudo-code.
- ☐ Je connais les trois parties d'un algorithme : en-tête, déclarations, corps entre Debut et Fin.
- ☐ Je sais distinguer une variable d'une constante.
- ☐ Je connais les cinq types de données : Entier, Réel, Caractère, Chaîne, Booléen.
- ☐ Je sais choisir le bon type pour une donnée (ex. une moyenne $\rightarrow$ Réel).

- ☐ Je sais utiliser Lire pour une entrée au clavier.
- ☐ Je sais utiliser Ecrire pour afficher un message ou un résultat.
- ☐ Je sais écrire une affectation avec le symbole $\leftarrow$ (jamais = en pseudo-code).
- ☐ Je sais que l'affectation remplace l'ancienne valeur de la variable.
- ☐ Je sais rédiger un algorithme complet sur une situation de la vie courante (calcul d'une moyenne, d'un montant, d'un périmètre...).
- ☐ Je sais justifier ma démarche et tester mon algorithme avec des valeurs concrètes.

**Signé • OnBuch+**Ludovic A.
CONFORME AU PROGRAMME OFFICIEL MINESEC**OnBuch+**